Company Supervisor

I authorize the student to submit her internship report for a defense.

Signature and stamp



ESPRIT Supervisor

I authorize the student to submit her internship report for a defense.

Signature

Mayel Jakroumi

Dedications

I dedicate this work

To my beloved parents Mohamed Najib and Amel,

There is no dedication that will convey what they deserve. Throughout my journey, they were the root of my bravery and patience. I will be eternally grateful to them for all of their encouragement in allowing me to pursue my studies in the best possible conditions. May God keep them safe and bless them with good health, peace, and long lives.

To my dearest siblings Sarra , Amor and Zin

For their reinforcement and support. I devote this work to them and wish them nothing but peace and prosperity in the future. May God bless them, keep them safe, and help them achieve all of their dreams.

To my loveliest Nephews Lynn and Jad

As a token of my devotion, admiration, and attachment to them. I wish them the best of luck in the future.

To my coolest friends Nesrine and Manel

Thank you for being a ray of sunshine even on my darkest days. I am blessed to have friends like you in my life. Please continue to be the weirdest mates.

Nessrine HAFI

Acknowledgements

At the closure of this work, I would like to express my heartfelt gratitude to anyone who has contributed to its completion, either near or far.

Softifi, the organization that accepted me, has my profound gratitude.

I'd like to express my appreciation to Mrs. ***Manel Takrouni*** *for her guidance, assistance, and encouragement, all of which were extremely helpful in the writing of this report.*

I'd like to convey my gratefulness to ***Mr. Wael Bouslama*** *and* ***Miss. Rabiaa Ghoubentini****, my Softifi supervisors, for their guidance, responsiveness, and support, both of which were extremely helpful in the creation of this project.*

It gives me great pleasure to thank all of the members of the jury, especially my rapporteurs, for agreeing to judge this work and for taking the time to read it.

I'd also like to express my sincere gratitude to all of my ESPRIT teachers for providing me with excellent instruction during my education journey at this prestigious institution.

Contents

General Introduction **1**

1 Project Context **3**

1.1 Host Organization : Softifi 4

1.1.1 Company presentation 4

1.1.2 Company Services 4

1.2 Project Scope 5

1.2.1 Motivation 5

1.2.2 Issues 6

1.2.3 Study and criticism of the existing 6

1.2.4 Proposed Solution 7

1.3 Methodology 7

1.3.1 Comparison of development methodologies 8

1.3.2 Agile Methodology 8

1.3.3 The Chosen method: SCRUM 9

1.3.4 Project Team 10

1.3.5 Project Planification 10

2 Stat of the Art **14**

2.1 DeVops 15

2.1.1 Traditional software development process 15

2.1.2 Introduction to DeVops 15

2.1.3 DeVops best practices and advantages 16

2.2 Continuous integration and Continuous delivery 16

2.2.1 Definition of CI/CD 16

2.2.2 CI/CD steps 17

2.2.3 Benefits of CI/CD 18

2.3 Virtualization 19

2.3.1 Overview 19

2.3.2 Containerization 19

2.3.3 Container Orchestration 21

2.4 Software testing . . . 23
2.4.1 Overview . . . 23
2.4.2 White box tests . . . 23
2.4.3 Black box tests . . . 23

3 Comparative study of DeVops tools 25
3.1 Comparative study strategy . . . 26
3.2 Source Code Management . . . 26
3.2.1 Selection criteria . . . 27
3.2.2 Overview of source code managers . . . 27
3.2.3 Comparison and synthesis . . . 27
3.3 Continuous integration servers . . . 28
3.3.1 Selection criteria . . . 28
3.3.2 Overview of IC servers . . . 28
3.3.3 Comparison and synthesis . . . 29
3.4 Software quality code tools . . . 29
3.4.1 Selection criteria . . . 29
3.4.2 Overview of Quality code tools . . . 30
3.4.3 Comparison and synthesis . . . 30
3.5 Monitoring Tools . . . 31
3.5.1 Selection criteria . . . 31
3.5.2 Overview of Monitoring tools . . . 31
3.5.3 Comparison and synthesis . . . 32

4 Analysis and Conception 33
4.1 The project's objective . . . 34
4.2 Requirements specification . . . 34
4.2.1 Actors identification . . . 34
4.2.2 Functional needs . . . 34
4.2.3 Non-functional requirements . . . 35
4.3 Conceptual modeling . . . 35
4.3.1 Presentation of the UML language . . . 36
4.3.2 Overview of UML Diagrams . . . 36
4.3.3 Overview of BPMN . . . 36

4.3.4 Modeling tools used . . . 36
4.4 Analysis model . . . 37
4.4.1 Use case Diagram . . . 37
4.4.2 BPMN diagram of the global process . . . 38
4.5 Detailed conception model . . . 41
4.5.1 Tools used by the company . . . 41
4.5.2 Environment Workflow . . . 41
4.5.3 Kubernetes Architecture . . . 44
4.5.4 Refined use case Diagram . . . 45
4.5.5 Sequence Diagram . . . 46
4.5.6 Componenet Diagram . . . 48
4.5.7 Deployment Diagram . . . 48

5 Implementation And Realization 50
5.1 Sprint 1 : Preparing the working environment . . . 51
5.1.1 Hardware . . . 51
5.1.2 Presentation of the Magento project . . . 52
5.1.3 DeVops Tools used . . . 52
5.2 Sprint 2 : Creating a Jenkins pipeline job . . . 54
5.2.1 Jenkins Installations . . . 54
5.2.2 Jenkins Global Configuration . . . 54
5.2.3 Plugin configurations . . . 55
5.2.4 Jenkinsfile . . . 57
5.2.5 Creation of Jenkins projects . . . 57
5.3 Sprint 3 : Preparing tests for the project . . . 58
5.4 Sprint 4 : Installing and integrating Sonarqube . . . 59
5.4.1 Jenkins with Sonarqube: . . . 59
5.5 Sprint 5 : Dockerazing the project . . . 60
5.6 Sprint 6 : Deploying the project using Kubernetes . . . 61
5.7 Sprint 7 : Monitoring . . . 62
5.8 Sprint 8 : Integrating all the tasks in the pipeline . . . 63

General Conclusion 69

Bibliography 76

List of Figures

1.1 Services offered by Softifi 5
1.2 Division of each scrum roles 10
1.3 Grantt Diagram 12

2.1 steps of CI/CD 17
2.2 Virtual machines vs Containers 20
2.3 Architecture of an orchestrator 21
2.4 Kubernetes Cloud provider 22

4.1 General use case of the project 37
4.2 BPMN of the project 39
4.3 Prototype of Stage Developement 42
4.4 Prototype of Stage Staging 43
4.5 Prototype of Stage Production 44
4.6 Overview of Kubernetes cluster 44
4.7 Interaction of Kubernetes services 45
4.8 Refined use case Diagram 46
4.9 Sequence Diagram for "Build pipeline" 47
4.10 Component Diagram 48
4.11 Deployement Diagram 49

5.1 VM specifications 51
5.2 Tajer pro home interface 52
5.3 Tools used in the DeVops workflow 53
5.4 Login interface in Jenkins 54
5.5 Jenkins plugin manager interface 54
5.6 Email notification setting interface 55
5.7 Slack plugin setting interface 55
5.8 Jira login interface 56
5.9 Jira server setting interface 56
5.10 Bitbucket webhook setting interface 57
5.11 Freestyle job 'Selenium' used in our project 58

5.12 Pipeline job 'Tajer Pro' used in our project 58
5.13 Tajer pro feedback interface 59
5.14 Sonarqube login interface 59
5.15 Sonarqube plugin setting interface 60
5.16 Sonarqube scanner setting interface 60
5.17 VM memory before docker 61
5.18 VM memory after docker 61
5.19 overview of a set of pods 61
5.20 Bitbucket webhook setting interface 62
5.21 grafana interface of the Kubernetes cluster 63
5.22 Overall view of the completed pipeline 63
5.23 Overall view of the completed pipeline 64
5.24 Code Coverage result in Jenkins interface 64
5.25 Report of Magento code in Sonarqube 65
5.26 Output of building dockerfile 66
5.27 Output of running the docker containers with docker-compose 66
5.28 Result of a generated functional test 67
5.29 working pods in google cloud platform 67
5.30 Email sent by Jenkins 68

List of Tables

1.1 Table comparing Traditional and Agile methodologies 8
1.2 Table comparing different types of Agile methods 9
1.3 List of sprints with description 11
1.4 The allocation of tasks During the internship 12

3.1 Continuous Integration Tools Comparison Table 28
3.2 Continuous Integration Tools Comparison Table 29
3.3 Quality Code Tools Comparison Table 30
3.4 Monitoring Tools Comparison Table 32

General Introduction

Technology, the 21st-century buzzword, has become such an integral part of our lives that it has become a requirement rather than a right to the point that, if it fails to exist, we cannot understand how our lives will work.

Technology has provided people with the means to learn, improve and apply their expertise and skills to devices and machines; hence make advancements towards better manufacturing and more productive processes.

With the introduction of technology, we have fundamentally changed the way we live, function, and play. Technology, a medium that empowers the individual and community, has given us many advantages, not least of all, greater productivity, and improved inner-connectedness.

As the most leading kind of technology, Software made a critical feature of contemporary society. It is not only required to be of good quality, but also to accommodate the evolving needs of the customer within the allotted budget and timetables. In order to ensure high-quality software goods, many software production processes have evolved over the years.

On the one hand, Plan-driven techniques are generally applied to large-scale or mission-critical projects, where specifications are not likely to change significantly over their life cycle.

In the other hand, using agile approaches, many software applications are better developed because they allow more flexibility to adjust specifications, which has always been the case in the changing world in which they work.

However, agile approaches have their drawbacks. By incremental development, they promise to produce applications quicker and to guarantee that it serves the needs of customers by focusing more heavily on consumer and client input than on plan-driven approaches. Nevertheless, Rapid delivery of software requires not only software development tasks but also operational activities, such as release implementation, the configuration of production environments, and management of the production environment in order to ensure high quality, such as high availability and efficiency.

In an effort to solve these challenges, DevOps has arisen by placing the primary emphasis on cooperation between development and operations, while automating as many procedures as possible before consumers can see a shift in the program. Generally, key advantages include

quicker and more regular launches through automation, expanded exchange of information through more efficient coordination and connectivity, as well as better offerings through an improved capacity to respond to evolving requirements.

There are also difficulties in the practice of DevOps, considering the benefits that DevOps has claimed to offer, such as lack of instruction on how to practice DevOps, lack of proper concept for DevOps, use of various development environments that present barriers for the introduction of continuous practices.

Still, numerous obstacles in the practice of DevOps are correlated with the technologies being used to simplify the activities of development and operation. The wide availability of tools makes it difficult to know which ones are suitable, to begin with for each project.

It is within this context that the current project, which entails the establishment of a continuous integration pipeline, aims to address many problems within Softifi in this context.

The plan envisioned in this report is structured in five chapters and takes a phased approach. The first chapter, titled "Project Context" gives a brief overview of the host company and the project's overall structure. It discusses the analysis of the current condition and the proposed solution. It concludes with a discussion of the measures taken to ensure that our work is completed efficiently.

The second chapter, "State of the Art," outlines the document's core theoretical ideas and numerous concepts. The third chapter, "Comparative study of DevOps tools" performs a comparative analysis of the most used DevOps tools on the market and presents the best tool in each category.

We first describe the functional and non-functional requirements of our structure in the fourth chapter, "analysis and design." Second, we deconstruct the design process using UML and BPMN diagrams.

Finally, the fifth chapter portrays the projects that were completed in order to complete this project. This report concludes with a general conclusion that summarizes the outcomes of the work performed, as well as the future directions for progress and contributions from a know-how standpoint.

Chapter 1

Project Context

Plan

1 Host Organization : Softifi 4

2 Project Scope 5

3 Methodology 7

Introduction

In this opening chapter, we will be introducing the host organization in which our graduation project was carried out. Then, we analysis the existence and we give our critique.Finally unveil the proposed solution as well as the working method used throughout the implementation of the project.

1.1 Host Organization : Softifi

1.1.1 Company presentation

Our Project was carried out by the firm "Softifi" . It is a Tunisian company founded in 2017 and it offers a range of Information Technology services designed for different businesses.

Softifi [1] provides a range of programs. Building ERP platforms, outsourcing, e-commerce solutions, and delivering a world-class consumer experience through robust digital marketing services are just a few of the services "Softifi" provide.

1.1.2 Company Services

Softifi is composed of various services such as:

- E-commerce: creating and delivering fast-performing e-commerce websites with an ideally designed online store to ensure an engaging shopping experience that is tailored to the preferences of the target audience and industry standards. Also, it is responsible for all aspects of its partners' e-commerce operations, including store management, payment processing, and ongoing support and maintenance.

- Web and Mobile Development: Integrating aesthetics and functionalities to build world-class websites and applications using the latest technologies to ensure various qualities like Seo Friendly, High security, Mobile responsive, etc.

- Digital Marketing: Providing analysis-based digital marketing services Based on the actual situation of each customer, such as solid branding and creative re-branding are offered to ensure that they achieve their final strategic targets.

The various services offered by Softifi are summarized in the following figure:

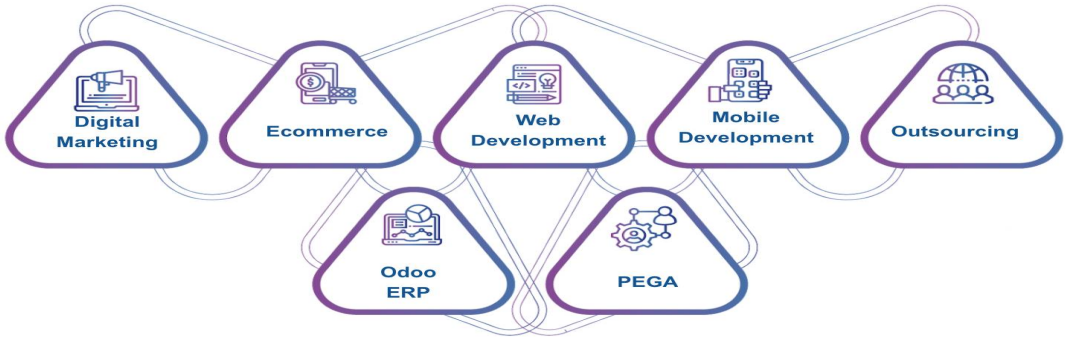


Figure 1.1: Services offered by Softifi

1.2 Project Scope

Placing a project in its larger context assists in not just recognizing the issue, but also identifying the need and proposing a solution to satisfy the need. therefore , We will start by explaining our motivation, then the issue that we want to solve, and finally our proposed solution just after the study of the existing.

1.2.1 Motivation

Since technology is evolving too rapidly in today's world, technology companies try to get their product to market as soon as possible in order to make a huge profit and address the concerns of many people. Any technology software industry requires a strong development team to produce the product and a strong operational team to deploy, maintain, and support the product and its customers.

Accordingly , they started integrating DeVops(Development Operations) philosophy into their workflow and immersed in Continuous integration and delivery to ensure that their services are delivered faster and with a high quality.

Therefore, Softifi decided to engage in this new world of DeVops and transform its E-commerce project using Magento into fully automated ones to guarantees the reduction of wasted time caused by conflicts between developers and operators and to integrate the entire workflow into a simple process that includes building, testing, quality check, deployment and finally monitoring.

1.2.2 Issues

Since Softifi is growing as an IT Company, the number of clients has increased. Therefore, the company started getting more demands. That's why they are considering expanding their team and seeking ways for a more speedy and effective process to deliver their services. Unfortunately, the use of traditional methods always triggers a lot of challenging issues.

Firstly, recruiting new developers requires installing and setting up their work environment in their operating system. Generally, it takes time since various errors and misconfiguration can occur during the install. And as the company requires all their employees to work in the same environment, they need to focus on specific versions.

Secondly, we need to ensure fast delivery of the code while ensuring that it matches specific qualities. But when the number of tasks increases, it's challenging to guarantee quality, and often tests get ignored. Therefore, the clients get disappointed by the poor quality.

Last but not least, high-end resources are needed so we can present recruits a suitable working environment to deploy their services and guarantee for their clients efficiency and security.

Moreover, is it possible for raising companies to eliminate all those problems and find a process that can help them gain time and assure quality while using a few resources?

1.2.3 Study and criticism of the existing

Since Softifi is upgrading their Magento version, a lot of variables changed. Consequently, the previous configurations are not adapting. Currently, the DeVops team uses Jenkins for continuous integration but using old ways for task scheduling, which isn't the best practice, particularly when operating on complex projects with various features to be automized since it can not be backed up nor restored. In addition, the majority of the jobs are executed manually by team members.

Additionally, when the automatized jobs break, there's no notification to report to employees concerned about the error.

Moreover, Softifi provides the newcomers with documentation to guide them to install the platform into their PC. Since Magento requires multiple services and applications, it can take a lot of time and minimum computer resources requirement, which costs the company a remarkable amount of funds.

1.2.4 Proposed Solution

After researching the issues cited above about the techniques used in Softifi, we discovered that multiple DeVops-inspired tools solve it. As a result, we decided to change the whole traditional process that we used to work with, making sure it adapts to the latest version of Magento. Here are the main changes that will be featured in our project :

- Since most of the procedures are running manually, we will put the Workflow (Dev - Stage - Prod) of the project in a pipeline job that will be triggered by each git push. It will assist us with continuous integration / continuous deployment. And inside the pipeline, we will make sure that the project will go through various types of tests and sending notifications in case of failure to ensure reliable code quality. In our case, we will be using Jenkins as the automation tool and Bitbucket for our version control. As for testing, we have three kinds: unit and integration using PHPUnit and Selenium for the functional tests. Also, we will use Sonarqube to guarantee code quality. And for notifications, Jira tickets will be created in case of failure during any stage in the pipeline.

- Containerizing Magento's software bundle (Linux, Nginx, Mysql, Php) into less resource-consuming containers to help newcomers avoid the long old process of install and reduce the use of computer resources such as the CPU, RAM ... Therefore, we will be using Docker for containerizing our services but also we will need a maestro that will help us orchestrate the containers, ergo, Kubernetes.

- We also need to monitor our PC and, how much resources are consumed to be alerted in case of a mal-function.

1.3 Methodology

Each software development team's primary concern is to complete the project within the specified time frame. Lackluster specification and unexpected changes in requirements are two of the most common issues encountered during software development. This will have an effect on not just the development team, but also the amount of time spent on project execution and delivery delays. Adopting an appropriate methodology is more than necessary in order to complete this work in an organized and rigorous manner. To do so, we'll first conduct a comparison of agile and traditional methodologies.

1.3.1 Comparison of development methodologies

In order to Choose the fitting methodology for our project, we presented a comparative table of the most know methodologies bellow.

	Agile	Traditional
User Requirement	Iterative acquisition	Detailed user requirements are well defined before coding/implementation
Rework costs	Low	High
Development direction	Readily changeable	Fixed
Testing	On every iteration	After coding phase completed
Customer involvement	High	Low
Qualities required for developers	Basic knowledge	Nothing in particular
Management style	Command and control	Leadership and Collaboration

Table 1.1: Table comparing Traditional and Agile methodologies

1.3.2 Agile Methodology

The agile approach was chosen because it was able to break away from the rigidity of the traditional one, particularly in terms of adapting to changing user needs.

The agile methodology has been proven on a variety of approaches, including Rapid Application Development, Extreme Programming, and Scrum, which we selected for our project.

In the Table 1.2 that follow, we'll go present a comparative table of Agile methods and their specifics.

	Extreme Programming	Scrum	Adaptive Software Development
Development implementation	Iterative Increments	Iterative Increments	Iterative
Iteration period	One to six weeks	Two to four weeks	Four to eight weeks
Project teams	Less than twenty team member	All sizes	Five to nine team member
Project size	Small projects	All types of projects	Small projects
Customer involvement	Customer involved	Through the role of product owner	Through releases
Advantages	-Open workspace -Customer as part of Team -Well defined best practices	-High level of Communication and Collaboration	-Development of high risk components first -Importance of learning cycle
Disadvantages	-Weak Documentation -Lack of discipline -Customer presence is mandatory	-Weak Documentation	-Poor documentation -Poor control over project

Table 1.2: Table comparing different types of Agile methods

We'll go through the details of the chosen method, as well as why we choose it, in the sections that follow.

1.3.3 The Chosen method: SCRUM

Scrum is undoubtedly one of the most popular agile methodologies because it offers a series of concrete measures that have aided businesses all over the world for a long time.

The idea behind this approach is to keep the team focused on the functionalities that need to be implemented in an iterative manner. As a result, the project is broken down into practical modules that will be created, reviewed, and distributed in sprints. Each sprint has a goal in mind, and the functionalities that will be incorporated in this process are determined by that goal. For a better understanding of this methods, we ought to understand the following keyword:

- **Scrum master:** Is the team member who is in charge of handling the process and ensuring the team's motivation as well as the success of the members' teamwork.
- **Product owner:** Is a project manager in agile mode. He serves as an interface between the technical team, the marketing team and the customers. Also primarily responsible for the design of the product.
- **Team members:** Are developers, designers, and testers within a team who do all the actual work of building, testing, and releasing the product.
- **Sprint Backlog:** The sprint backlog is a list of tasks that the Scrum team has listed to be accomplished during the sprint.
- **Daily meeting:** Is a daily check-in that helps you to assess what has been accomplished, the challenges faced, and the goals set for the day.

Along with the presentation of the main concept of the Scrum method, we will introduce you to all of the participants who were involved in our project in the following section.

1.3.4 Project Team

In order for our project to be established using the scrum method, we divided the main three roles which will be shown in figure 1.2.



Figure 1.2: Division of each scrum roles

1.3.5 Project Planification

In project management, project planning is an inevitable phase. It allows you to plan out the tasks you'll do, set goals, and keep track of the tools you'll use to reduce risks. We started by breaking down the project into tasks and then creating a Gantt chart.

1.3.5.1 Project planification in sprints

The project is divided into eight sprints, with each sprint's title being a function of our project backlog.

N°	Sprint title	Description
1	Preparing the working environment	Prepping the virtual machine and installing Magento. Also, learning about available DeVops tools and choose the most beneficial one that can adapt accordingly to our working environment.
2	Creating a Jenkins pipeline job	Creating Jenkinsfile to outline to pipeline into stages for the different tasks.
3	Preparing tests for the project and executing them inside the pipeline	Writing a unit and integration for a Custom module in the Magento project and integrating them inside a stage in the Jenkins pipeline.
4	Installing and integrating Sonarqube	Installing Sonarqube inside our virtual machine and configure it to analyze our code with specific settings and finally add it to our project.
5	Dockerazing the project	Containerizing the required services used to run Magento into Docker containers by writing Dockerfile and docker-compose and deploying it.
6	Deploying the project using Kubernetes	Exposing the application that operates on a series of pods using the google Kubernetes engine.
7	Monitoring	Supervising the used virtual machine and the Kubernetes cluster where our application is deployed.
8	Integrating all the tasks in the pipeline	Setting all the previous tasks inside the pipeline to finish the whole DeVops workflow.

Table 1.3: List of sprints with description

1.3.5.2 Chronogram

This work was completed over a six-month cycle. The following table and Gantt chart depict the mission distribution during this time period:

Tasks/Activities	Starting date	Duration	Ending date
Discovering Magento	01/10/2020	12 days	16/10/2020
DeVops Training	19/10/2020	20 days	13/11/2020
Preparing the working environment	16/11/2020	10 days	27/11/2020
Creating a Jenkins pipeline job	30/11/2020	5 days	04/12/2020
Preparing tests for the project and executing them inside the pipeline	07/12/2020	15 days	25/12/2020
Installing and integrating Sonarqube	28/12/2020	14 days	15/01/2021
Dockerazing the project	18/01/2021	20 days	12/02/2021
Deploying the project using Kubernetes	15/02/2021	20 days	12/03/2021
Monitoring	15/03/2021	5 days	19/03/2021
Integrating all the tasks in the pipeline	22/03/2021	5 days	26/03/2021

Table 1.4: The allocation of tasks During the internship

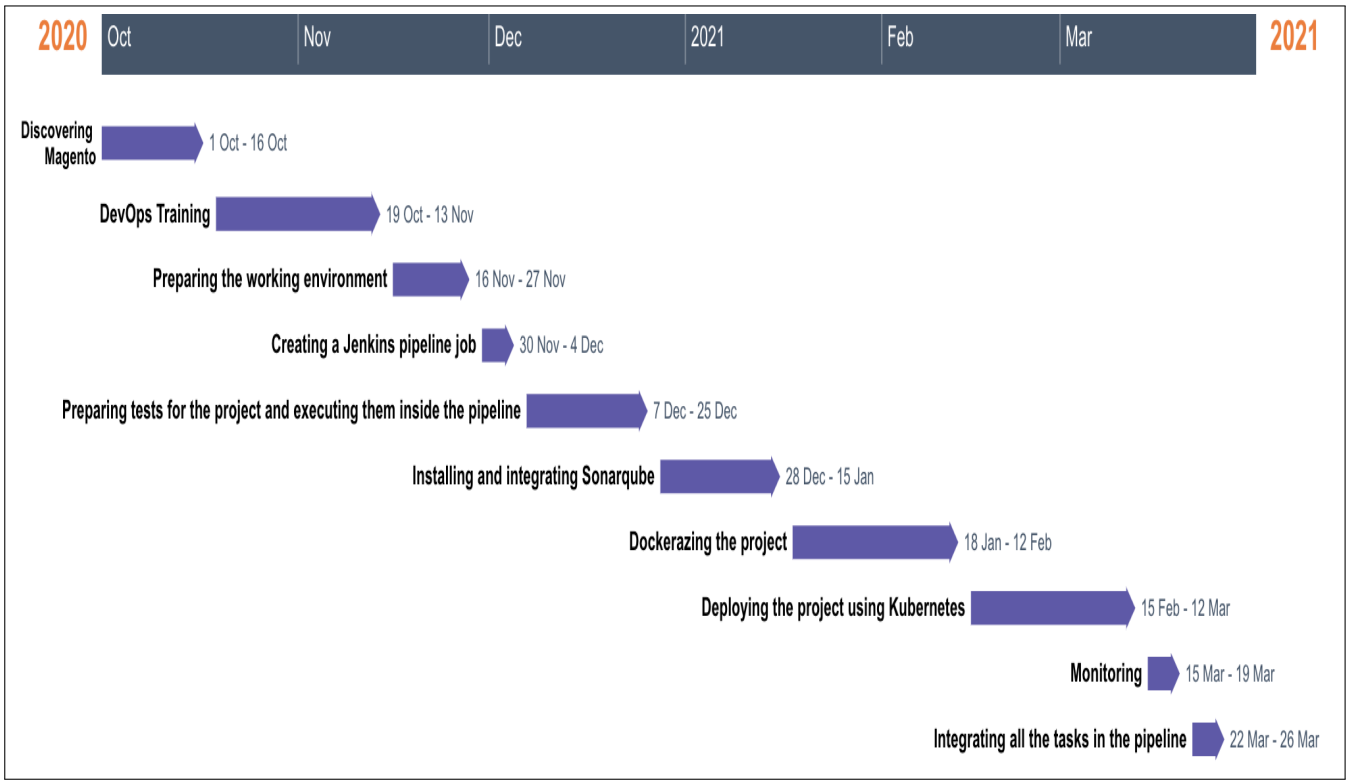


Figure 1.3: Grantt Diagram

Conclusion

This chapter served as an introduction to the host organization as well as a general overview of the project. We were also able to identify the requirements of the potential approach to be applied after doing an analysis of the current situation.

Following the goals, the next chapter will focus on incorporating the various concepts required for a thorough understanding of our project.

Chapter 2

Stat of the Art

Plan

1 **DeVops** 15

2 **Continuous integration and Continuous delivery** 16

3 **Virtualization** 19

4 **Software testing** 23

Introduction

The aim of this chapter is to explain the different concepts explored in this report. First, we'll go over what DeVops is and what the principles are. The definition of continuous integration would then be defined, as well as its components and benefits. Then we'll go over the virtualization and orchestration principles. Finally, we'll go into software testing in general.

2.1 DeVops

Before starting to present DeVops, it is important to first address the traditional development process in order to identify the issues that led to its appearance.

2.1.1 Traditional software development process

Traditionally, there are two groups of explicitly defined key players within an information system: development teams on the one hand, and system administration teams on the other.

Both physically and managerially, these two groups are usually divided. Each is constrained by budget and time, and each attempts to accomplish its own goals.Unfortunately, as IT progress has accelerated over the last two decades or so, these goals have continued to conflict since they have overlapping priorities.

Development is mainly concerned with the production of new systems and software and ensuring that consumers can access them as soon as possible. Operations, on the other hand, take a different approach, focusing primarily on maintaining a fast, bug-free, and reliable system.[2]

This causes tension, if not outright confrontation, which may have huge consequences for the company's viability and image. The birth of DeVops can be traced back to the recognition of these issues.

2.1.2 Introduction to DeVops

DeVops is a recent movement that arose from the collision of two old trends: "agile system administrations" or "agile operations," and "understanding the importance of teamwork between development and operations staff at all stages of the DeVops life cycle." [3]

This movement began in 2010 and is marked by the automation and monitoring of all phases of software development, including development, testing, delivery, and production for infrastructure deployment and maintenance.

2.1.3 DeVops best practices and advantages

There are a few main DeVops activities that will help companies innovate more quickly. One important technique is to update the program on a regular basis, but with small updates.
The use of such updates helps to reduce the risks that come with each implementation. Since the last deployment that triggered the error can be determined, developers are more likely to encounter bugs.
DeVops organizations use Micro-services architecture to make their systems more scalable and allow faster innovation. The micro-services architecture breaks down massive, complex structures into small, self-contained projects. Each service is associated with a particular function and operates independently of the other services and the application.
However, the use of micro-services in conjunction with a higher delivery frequency results in a significant increase in the number of deployments, which may cause technical challenges. Continuous integration and continuous delivery practices address these issues and enable companies to deliver easily, efficiently, and securely.
The following sections will concentrate on the final point in DeVops best practices, which is continuous integration / continuous delivery (CI/CD) .

2.2 Continuous integration and Continuous delivery

2.2.1 Definition of CI/CD

Continuous integration (CI) is a coding philosophy and collection of practices that encourages development teams to make small changes and frequently check code into version control repositories. Since most modern applications necessitate writing code across several platforms and resources, the team requires a way to implement and verify the changes.
The technical goal of CI is to provide a standardized and automated process for developing, packaging, and testing applications. Teams are more able to commit code updates more often as the integration process is consistent, which contributes to greater teamwork and software efficiency.
Where continuous integration ceases, continuous delivery begins. CD automates application delivery to specific technology environments. Most teams operate in areas other than production, such as development and testing, and CD means that code updates are automatically pushed to those environments.

The environment-specific parameters that must be packaged with each delivery are processed using CI/CD software. When applications are deployed, CI/CD automation makes all required service calls to web servers, databases, and other resources that might need to be restarted or follow such procedures.

Since the goal of continuous integration and continuous delivery is to deliver high-quality software and technology to customers, continuous testing is needed. Continuous testing is often applied as a series of automatic regression, performance, and other tests run via the CI/CD pipeline.

Setting up a CI/CD framework entails automating a series of processes and integrating a collection of tools. In the following sections, we'll go through these measures and tools in greater depth.[4]

2.2.2 CI/CD steps

The steps of Continuous Integration / Continuous Delivery are described in figure 2.1.

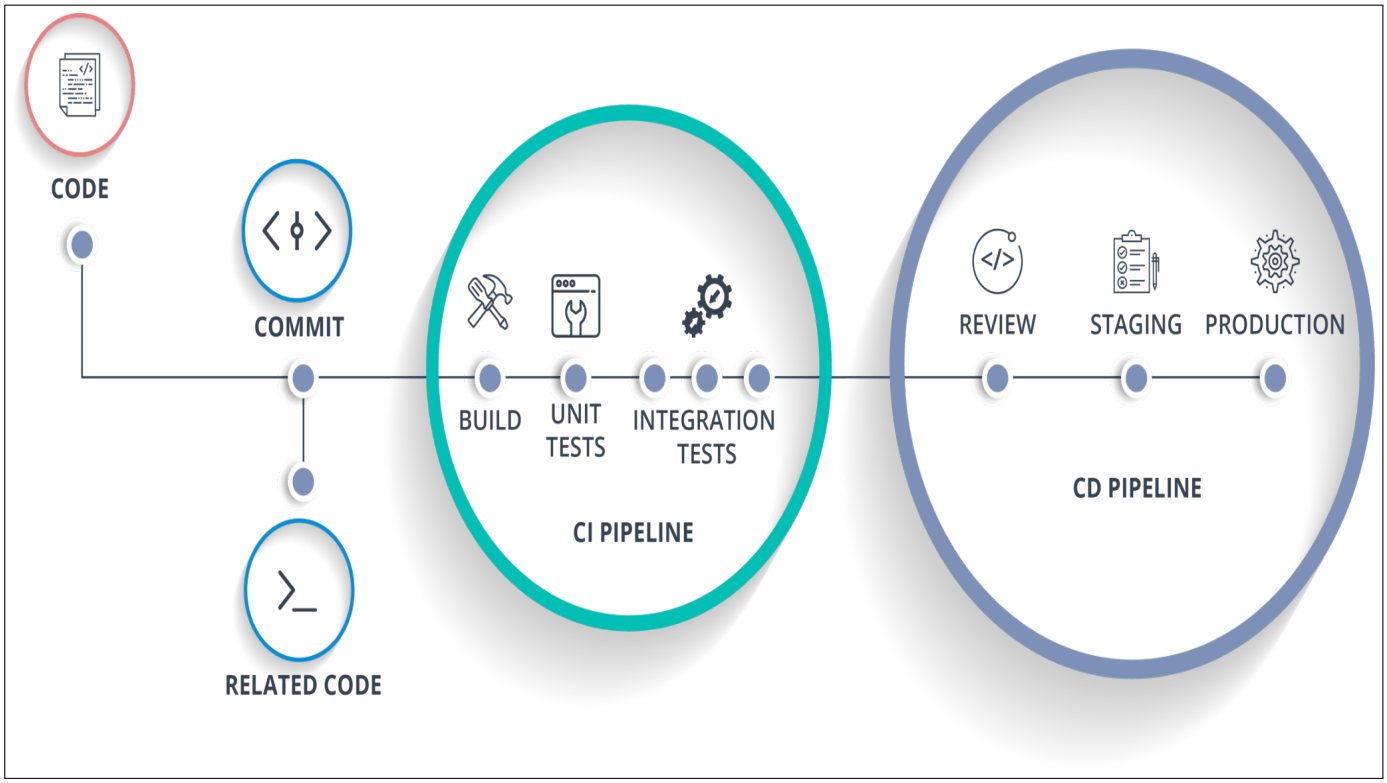


Figure 2.1: steps of CI/CD

These steps are based on the following activities:

① Each developer works on his or her own machine. First and foremost, he would pull the most recent version of the program from his version control system (such as git, svn). Afterward, once he has made a significant change to the code, he will push his work toward his version control system also known as a "commit".

② In the second step continuous integration, a workflow engine controlling a branch of a source code repository can trigger an order to a build script, A configuration file, to create the code and execute the unit and integration tests in a managed environment until the code is moved to that branch.

③ The next step is continuous delivery (CD), which involves deploying the program code in a staging environment that is a copy of the production system and performing more functional tests. You can either provision and configure a dynamic environment of committed infrastructure and configuration code for developing and deploying software application, or you can provision and configure a static environment that has already been set up for testing.

2.2.3 Benefits of CI/CD

CI/CD automates the process of bringing the product to market ahead of the competition, as well as releasing new updates and bug fixes to keep your consumers happy.
In fact, integrating such a process helps companies to [5]:

- **Better code quality:** One of the most key evolutionary advantages of CI/CD is that it improves overall code quality. This makes it easier to share stable versions that are free of crucial bugs more often, and poor code is less likely to make it to market. That's how implementing CI/CD reduces the chances of buggy code making it too far into production.

- **Accelerated Release Cycles:** To maintain the code in a release-ready environment, CI/CD keeps merging and deploying it to production-like systems on a regular basis. This also allows the company to create a consistent delivery mechanism that follows the same steps for each change.

- **Enhanced Test Reliability:**Using CI/CD, you can increase test reliability to a great degree. Because basic and atomic modifications are added to the system, it helps developers or QAs to incorporate more important positive and negative checks for the changes.

- **Enhance developer productivity:** Continuous integration helps increase developer productivity by reducing the number of manual tasks they must perform. As a result, it reduces errors and bugs in releases to customers.

2.3 Virtualization

2.3.1 Overview

Virtualization is a computing mechanism that allows one or more physical machines to be emulated and run as virtual machines (VMs) on a server using a hypervisor. Each VM has its own operating system, which runs the applications it supports.
Despite the fact that the method is proven, we can easily see how each iteration of the host operating system and associated binary files is likely to result in VM duplication, wasting server memory and limiting the number of VMs supported by each server.
Many companies are turning to a new virtualization solution known as operating system virtualization, also known as containerization, to solve this problem.

2.3.2 Containerization

Containerization is a form of operating system virtualization in which programs are run in isolated user spaces known as containers, all while sharing the same operating system (OS) [6]. Since each "container" only includes the program and its related binaries, this strategy ultimately decreases resource loss. Instead of installing an operating system for each guest virtual machine, we use the same host OS for several containers.

2.3.2.1 What is the concept of a container?

A container is a standard software unit that packages programming and all of its dependencies so that the program can be migrated from one operating environment to another easily and efficiently[7].

Unlike server virtualization and virtual machines, the container does not have an operating system; instead, it is completely based on the operating system of the server on which it is built.

2.3.2.2 Comparison between a container and a virtual machine

Both solutions are feasible when it comes to setting up a test environment. Containers, on the other hand, have a significant benefit over virtual machines.

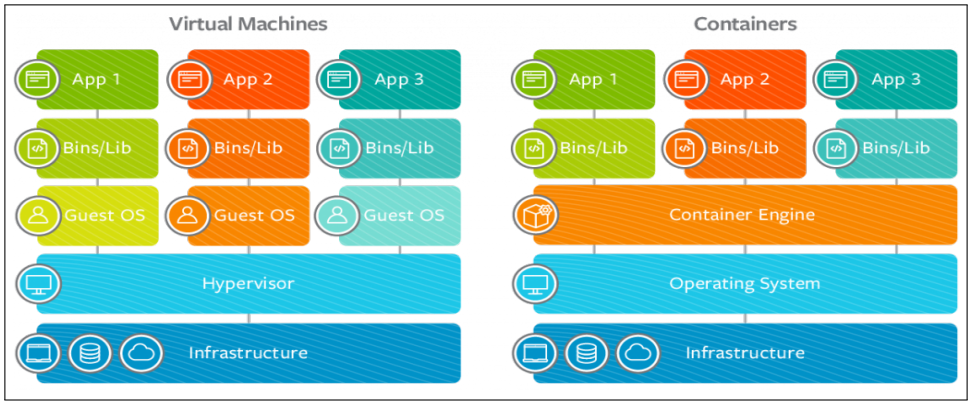


Figure 2.2: Virtual machines vs Containers

Since each program is freed from the load of an OS, containers are much smaller, easier to transfer or install, and simpler to backup or restore than virtual machines. Containerization was selected to prepare the test system for this project.
A containerization engine, which will act as a hypervisor, will be needed to build containers. We will use the well-known Docker platform in our project.

2.3.2.3 Docker

Docker is an open source software framework that allows you to build, install, and manage virtualized application containers on a common operating system (OS) using a set of tools[8]. This tool has the advantage of having various program versions to be designed for each container. As a result, you can install two incompatible programs on the same machine and also run two separate versions of the same program at the same time without having to think about dependencies.

2.3.2.4 Docker Terms

- **Docker Image:** Read-only template that includes a series of instructions for building a Docker container. It allows you an easy way to package up software and preconfigured server environments that you can use privately or openly with other Docker users[9].

- **Docker File:** A script/text configuration file that includes a series of commands and instructions that will be executed in the Docker environment in order to generate a new Docker image. This document is written in YAML, a popular human-readable Markup Language.[10].

- **Docker Compose:** Docker Compose is a Docker tool that allows you to build and run multi-container apps. Compose helps you to customize the application's services using a YAML file and then build all of the app's services from that configuration[11].

2.3.3 Container Orchestration

2.3.3.1 Definition

Container orchestration is the practice of automating all elements of container coordination and management, with an emphasis on container lifecycle management and their complex environments. It allows for the management and automation of a range of functions, including container provisioning and deployment, load balancing, redundancy and availability, resource allocation, and many more. In our case we will be using the well-known Kubernetes.

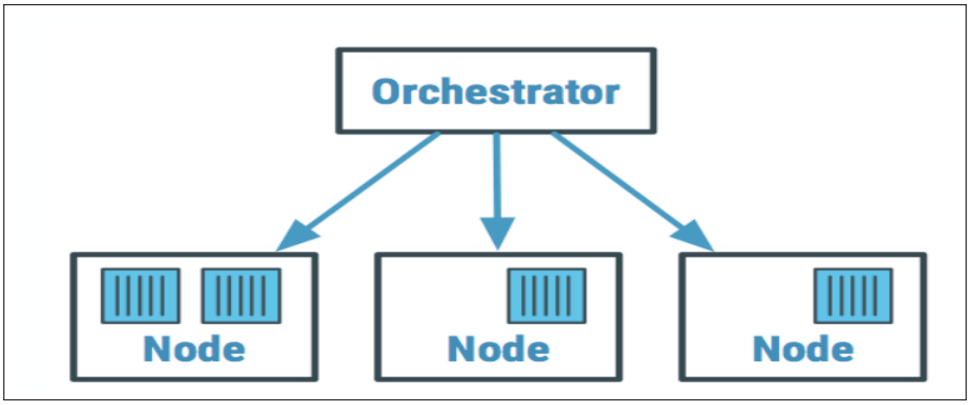


Figure 2.3: Architecture of an orchestrator

2.3.3.2 Kubernetes

Kubernetes was released by Google as an open source project in 2015. It automates multi-container application deployment and management at scale. It's a cluster-based architecture for operating and managing containerized software. It's a framework that uses a predictive, flexible, and high-availability approach to completely handle the lifecycle of containerized software and services.

Kubernetes is mainly compatible with Docker, but it can also be used with any container system that supports the Open Container Initiative's image formats and execution environments. Kubernetes can be used by anyone, anytime, due to its open source nature.

2.3.3.3 Kubernetes Core Concepts

- **Master node:** Multiple controllers are in charge of the cluster's health, replication, scheduling, endpoints, Kubernetes API, and communicating with the underlying cloud providers, among other things. In general, it ensures that everything is functioning well and looks after worker nodes.[12].

- **Worker node:** Runs the Kubernetes agent, which is in charge of running Pod containers via Docker, requesting secrets and configurations, mounting required Pod volumes, performing health checks, and reporting Pod and node status to the rest of the system[12].
- **Pod:** The smallest and most basic unit that you can build or deploy in the Kubernetes object model. It reflects a cluster process that is currently operating. It can accommodate one or more containers[12].
- **Deployment:** Provides declarative changes for Pods (such as the Pod template), such as the Docker image(s) to use, environment variables, the number of Pod replicas to run, names, node selectors, volumes, and so on.[12].
- **ReplicaSet:** At any given time, the controller ensures that a specified number of Pod replicas are running[12].
- **Service:** An abstraction that describes a logical collection of Pods as well as a policy for accessing them (determined by a label selector). It's typically used to expose Pods to other cluster services (via targetPort) or externally (using NodePort or LoadBalancer objects)[12].

2.3.3.4 Cloud provider

In Kubernetes, Cloud providers are a module that provides an interface for managing load balancers, nodes, and networking routes. A cloud provider is a company that provides businesses and/or individuals with cloud computing-based services and solutions. This service organization may offer virtual hardware, software, infrastructure, and other services that are rented and managed by the provider. Companies are increasingly interested in cloud services because of the cost, scalability, and accessibility benefits they provide. In our Case we will be using Google Cloud Platform(GCP)

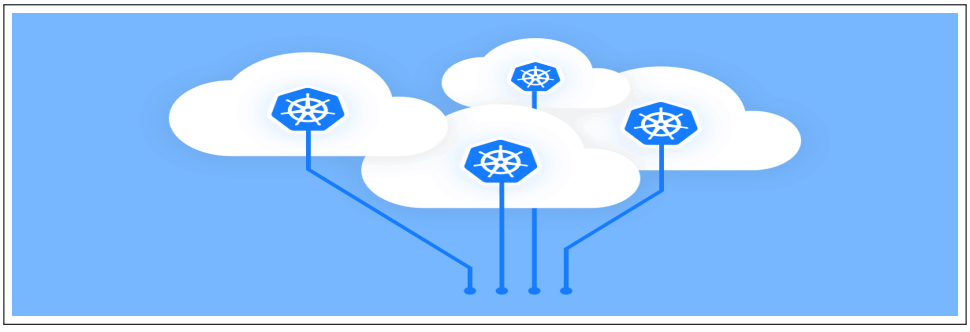


Figure 2.4: Kubernetes Cloud provider

2.4 Software testing

When we decide to configure and implement a continuous integration platform, It's a must to plan for the implementation of tests. Without testing, the perks of such an environment are severely limited.

2.4.1 Overview

A process of analyzing a program for flaws is known as software testing. Testing is the process of executing a set of test scenarios to ensure that both internal logic and external requirements are met. "White-box" and "Black-box" tests are the two major types of tests available.

2.4.2 White box tests

White Box Testing is a type of software testing in which the software's internal structure, design, and coding are analyzed in order to verify input-output flow and improve design, usability, and security. White box testing is also known as Transparent box testing and Code-based testing because the code is visible to testers[13].
This type contains the following tests:

- **Unit Testing:** A method of testing a unit, which is the smallest piece of code in a system that can be logically isolated. That is a function, a subroutine, a method, or a property in most programming languages.

- **Integration testing:** A type of testing in which software modules are integrated and tested as a unit. The goal of this level of testing is to find flaws in the way these software modules interact when they're integrated.

2.4.3 Black box tests

Black Box Testing is a software testing process that tests the functionalities of software applications without knowing the internal code structure, implementation details, or internal routes. Black Box Testing is a type of software testing that focuses on the input and output of software applications and is entirely driven by software requirements and specifications. Behavioral testing is another name for it[14]. The following are examples of black-box testing:

- **Functional Testing:** A form of software testing that verifies that the software framework meets the functional criteria. Functional checks are used to validate the performance of

a software program by supplying appropriate input and comparing the output to the functional specifications.

- **Non-Functional Testing:** A method of software testing that examines a software application's non-functional aspects (performance, usability, functionality, and so on). It's intended to assess a system's preparation based on nonfunctional criteria that aren't covered by functional testing.

Conclusion

This chapter explained the fundamental principles that aid in the comprehension and completion of our project. We covered the basics of DeVops and continuous integration / continuous delivery in the first two sections. The idea of containerization was then clarified, with an emphasis on Docker technology and Kubernetes.

We wrapped up this chapter with a quick overview of software testing. As a result, this chapter has offered a general understanding of the different topics covered in our project. The next chapter will provide a comparative analysis of the methods that can be used to complete this mission.

Chapter 3

Comparative study of DeVops tools

Plan

1 **Comparative study strategy** **26**

2 **Source Code Management** **26**

3 **Continuous integration servers** **28**

4 **Software quality code tools** **29**

5 **Monitoring Tools** **31**

Introduction

A CI/CD pipeline necessitates the selection of a set of tools. However, given the wide range of options available, selecting the winning mix remains a difficult challenge. To do so, a comparative analysis must be conducted to determine the right tools. The best tool must include features that are relevant to our project's requirements.We'll start with an overview of the comparative study's approach. Then we'll go over the approach we took and the specifics of the analysis we performed for each part of our continuous integration system. Finally, we'll show you the outcomes of the tools we chose.

3.1 Comparative study strategy

A major factor in the effectiveness of a continuous integration pipeline is selecting a streamlined mixture of DeVops tools that interact with each other. On the market, you can find a variety of tools. As a result, the decision is based on the criteria that must be taken into account.
We follow the steps below to better coordinate this analysis, which is a prerequisite for the pipeline implementation phase:

* Create a list of the most well-known tools on the market and pick the ones that benefit you the most to analyze and study.
* Establish a list of specific and detailed comparative criteria.
* Develop tool comparison tables based on criteria you've already settled on.

In what follows, we'll present a comprehensive comparative study for each tool group that closely resembles the approach we just mentioned.

3.2 Source Code Management

A source code manager is a prerequisite for every development firm. This tool helps you to store all of the source code and related files while keeping track of all of the changes made in a chronological order.

3.2.1 Selection criteria

We used the following criterion to choose the best source code management solution for our needs:

- **License:** it is free , paid .
- **Open Source:** The source code for the software is available to the public.
- **Version Controller:** Supported version control software.
- **Problem Tracking :** Without using any other software, keep track of project accidents.

After we've decided on the important set of comparison criteria. Bitbucket, Github and Gitlab were chosen as the top three source code management tools.

3.2.2 Overview of source code managers

3.2.2.1 Bitbucket:

Bitbucket is a commercial or free web-based software hosting and development management tool that uses the Mercurial or git version control systems. The above helps you to build an infinite number of private repositories, each of which can be accessed by up to five people [15].

3.2.2.2 Github:

Github is a hosted web repository tool that helps developers to store and share their code, whether publicly or privately. For projects, it offers collaboration, access control, wikis, and basic task management methods.

3.2.2.3 Gitlab:

Gitlab is a free and open source solution for hosting and managing git-based projects. It has a wiki, a bug tracker system, continuous integration, and continuous delivery features.

3.2.3 Comparison and synthesis

Based on the parameters defined and discussed earlier, Table 3.1 summarizes the comparison of the source code management software presented above.

	Bitbucket	GitHub	GitLab
Licence	Freemium	Freemium	Freemium
Open source	✖	✖	✔
Version controller	git , Mercurial	git	git
Problem tracking	Jira	✔	✔

Table 3.1: Continuous Integration Tools Comparison Table

For our project, we will be using Bitbucket as our version control tool. Github and GitLab are very well-known tools, but Bitbucket has a very interesting set of integration such as Jira since they belong to the same platform also it's freemium.

3.3 Continuous integration servers

Continuous integration program is a mechanism that helps developers to integrate and validate the code they're working on as a team any time they make an update, ensuring that bugs are caught early. This tool, when correctly installed, can automate repeated and repetitive activities during the project lifecycle, from project conception to execution in a production environment.

3.3.1 Selection criteria

We used the following parameters to pick an effective continuous integration tool:

- **License:** it is free , paid .
- **Integration:** The software's integration with other tools.
- **Documentation:** The richness of the documentation
- **Operating system :** Supported operating systems.

We wanted to compare Jenkins, Travis CI, and Gitlab CI because there are many continuous integration solutions on the market today.

3.3.2 Overview of IC servers

3.3.2.1 Jenkins:

Jenkins is an open source platform that is mostly used in the industry for continuous integration solutions. This software has a range of features and interfaces that make continuous integration,

as well as continuous delivery and deployment, much simpler.

3.3.2.2 GitLab CI:

GitLab CI is a componant of the well-known GitLab source code management framework. This tool provides continuous deployment and delivery in addition to continuous integration. Its setup is achieved via a YML format, similar to Travis CI.

3.3.2.3 TravisCI:

Travis CI is a free and open-source continuous integration platform. It integrates with a range of source code management tools, including Github. This tool can be customized by editing a YML file in the development project's root directory.

3.3.3 Comparison and synthesis

The comparison of the continuous integration servers discussed earlier is summarized in Table 3.2.

	Jenkins	Travis CI	GitLab CI
Licence	Free	Free	Freemium
Integration	⋆⋆⋆⋆⋆	⋆⋆⋆⋆	⋆⋆⋆⋆
Documentation	Adequate	Poor	Rich
Operating system	Linux , MacOS , Windows	Linux , MacOS	Linux

Table 3.2: Continuous Integration Tools Comparison Table

Jenkins is the most suitable technique for our situation, according to this table and the various details obtained in this report. It is simple to combine with other software and comes with sufficient documentation. Furthermore, this program has a large number of plugins and a free licence, giving it an advantage over rivals Gitlab CI and Travis CI.

3.4 Software quality code tools

To measure code quality, test coverage, and publish reports on the different metrics collected, quality measurement tools are used. These reviews are used to determine the amount of technological debt accumulated and to forecast possible mistakes.

3.4.1 Selection criteria

We used the following criteria to choose a suitable software quality code tool:

- **License:** it is free , paid .
- **Open source:** The source code for the software is available to the public.
- **Documentation:** The richness of the documentation
- **Supported languages:** It specifies how many languages the tool can analyze and scan.

3.4.2 Overview of Quality code tools

3.4.2.1 Sonarqube:

Sonarqube is an open source software tool that measures the accuracy of source code. Duplicate code, coding standards, unit checks, code complexity, and security bugs are all reported using this method.

3.4.2.2 Codacy:

Codacy is a code quality analysis tool that assists developers in creating high-quality applications quickly. Via static code review analysis and tracking of different measurements, the tool automatically detects issues.

3.4.2.3 Scrutinizer:

Scrutinizer is a tool for continuous inspection that assists developers in developing better applications. With basic and easy-to-understand code metrics, it is possible to consistently assess and track code quality.

3.4.3 Comparison and synthesis

Based on the criteria defined and explained previously, Table 3.3 shows a comparison of the software quality measurement tools previously presented:

	sonarqube	CODACY	Scrutinizer
Licence	Free	Paid	Paid
Open source	✔	✖	✖
Documentation	Rich	Rich	Very Rich
N° of supported language	27	7	30

Table 3.3: Quality Code Tools Comparison Table

We chose Sonarqube as a software quality control tool based on the results of this research. In one side, because of its open source nature and the fact that it is distributed under a free

license. But again, because of the vast number of languages supported and the scale of the community.

3.5 Monitoring Tools

Data is at the core of the decision-making when developing for stability or adopting reliable DeVops techniques. You won't know where to focus development efforts or improve operational practices until you closely track key indicators like uptime, network load, and resource use[16]. Fortunately, a number of monitoring tools are available to assist you in gathering and analyzing this data.

3.5.1 Selection criteria

To choose the appropriate Monitoring tool, we used the following criteria:

- **License:** it is free , paid .
- **Open source:** whether the source code for the software is available to the public or not.
- **Database:** The database used.
- **Cross-platform:** It designate whether the tool works across multiple types of platforms It specifies how many languages the tool can analyze and scan.

3.5.2 Overview of Monitoring tools

3.5.2.1 Grafana

Grafana is an open source solution for performing data analytics, retrieving measurements that make sense of large amounts of data, and monitoring our applications using pleasant personalized dashboards. Grafana integrates with a wide range of data sources, including Graphite, Prometheus, Influx DB, ElasticSearch, MySQL, PostgreSQL, among others[17].

3.5.2.2 Kibana

Kibana is an open source analytics and visualization tool that was created specifically to work with Elasticsearch. Search, View, and Handle data stored internally in Elasticsearch indices are the most common uses of Kibana. Advanced data processing and visualization of the data can be achieved with ease using charts, graphs, and maps[18].

3.5.2.3 Splunk

Splunk comes as a powerful platform that helps analyze machine data. Splunk will send you a panoramic view of what's going on with your machine-generated results. Splunk can also be used to classify past patterns and compare a variety of data sources, and it can be useful in a variety of ways[18].

3.5.3 Comparison and synthesis

	Grafana	kibana	splunk>
Licence	Free	Freemium	Paid
Open source	✔	✔	✖
Database	Variety of databases(by default sqlite3)	Elasticsearch	Mongodb
Cross-platform	✔	✖	✔

Table 3.4: Monitoring Tools Comparison Table

In our case, Grafana is the most suitable tool to monitor our project since it's free and open source. Also, it supports different kinds of databases and platforms.

Conclusion

We contrasted the tools we'll use in our continuous integration pipeline in this chapter. We began by deciding on a strategy. Then, we compared each tool type and put the tools together in a synthesis to determine which software was most appropriate. As a result, we were able to define the tools we'll use to build our final solution. The next chapter, "Analysis and Conception" will concentrate on the details of this solution's design.

Chapter 4

Analysis and Conception

Plan

1 **The project's objective** **34**

2 **Requirements specification** **34**

3 **Conceptual modeling** **35**

4 **Analysis model** **37**

5 **Detailed conception model** **41**

Introduction

This is the strategy that must be pursued to excel in any project: think before you act, prepare before you create, conceptualize first, create later. As a consequence, this chapter will concentrate on the measures involved in designing our approach.We'll begin by outlining the project's goals and requirements. The actors in the system will then be identified, and the modeling section of the solution will be detailed using UML and BPMN diagrams.

4.1 The project's objective

Using the practice of "Pipeline as Code," this project aims to implement a continuous integration / continuous delivery pipeline that supports source code analysis, automated test launching, Docker image creation, and Kubernetes deployment.

4.2 Requirements specification

The requirements analysis and specification process is a crucial step that helps us to identify the actors that engage with our system and to clearly define the work that needs to be done from both a functional and non-functional standpoint.

4.2.1 Actors identification

An actor is an entity that interacts directly with the system under study. It is a role that can be filled by an individual, a software, or a hardware. We recognize four actors in our CI/CD system:

- **Developer:** Is a person who has the right to change the source code for a particular purpose.
- **QA Engineer:** Is a person who configure code quality tool to display the result.
- **DeVops Engineer :** Is a user who administrate the code source and configure the workflow and DeVops features.
- **Jenkins:** Is the task-automation server for continuous integration.

4.2.2 Functional needs

This are the specifications for the system's input/output actions. They are the tasks that must be completed by the CI/CD pipeline. In fact, the following features must be available in our

system:

- Launching Jenkins pipeline automatically following a code push to Bitbucket.
- Building the project and running the tests automatically.
- Automation of Docker image creation.
- Deploying the application using docker containers in our local. host
- running functional tests using selenium.
- Deploying our application in Kubernetes cluster using google cloud platform as our cloud provider.
- Creating Jira tickets when the pipeline breaks.
- Monitoring our system.
- Notify when the pipeline ends by email and slack.

4.2.3 Non-functional requirements

The non-functional specifications are the guidelines that must be followed in order to guarantee the consistency of the planned mechanism and the future system's efficient maintenance. As a result, our continuous integration/ continuous delivery system must meet a number of requirements in order to have a consistent and effective solution:

- **Load Balancing:** This means that no single server is overburdened. Load balancing increases application responsiveness by distributing jobs equally.
- **High Availability:** Refers to a system's or component's ability to remain operational for an extended amount of time.
- **Performance:** The system components must be efficient in terms of execution and response time.
- **Conviviality:** The tool interfaces must be quick to grasp and control.

4.3 Conceptual modeling

Following the analysis of our system's functional and non-functional specifications, the next step is to convert them into a conceptual model.

4.3.1 Presentation of the UML language

UML (Unified Modeling Language) is a standardized modeling language that consists of an integrated range of diagrams that was generated to assist system and software development in defining, visualizing, designing, and reporting software system artifacts, as well as business modeling and other non-software systems[19].

4.3.2 Overview of UML Diagrams

In order to satisfy the needs of our solution, we have limited ourselves in this work to the following diagrams: the use case diagram, the component diagram and the deployment diagram.

- **Use Case Diagram:** This diagram shows the general and essential functions performed by the system.
- **Component Diagram :** A diagram used to demonstrate how different components in a system interact.
- **Deployment Diagram:** Deployment diagrams are widely used to represent a system's physical hardware and software.

4.3.3 Overview of BPMN

BPMN stands for Business Process Model and Notation, and it is a widely used modeling standard for business process modeling. It's frequently made to help with business process management, development, or reengineering. BPMN illustrates business processes from start to finish, illustrating the sequence of process operations as well as the exchange of information between participants[20].

4.3.4 Modeling tools used

We used two online software packages to model our solution: StarUml for UML diagrams and bpmn.io for BPMN diagrams. These two applications have a wide variety of features that make it easy to generate various types of diagrams.
We created an analysis model, a detailed design model, and a deployment model to design a complete architecture of our system. The handling of our approach would be plain and simple with these models.

4.4 Analysis model

4.4.1 Use case Diagram

A general use case diagram can be used to model the proposed system's functional activity. Figure lists the general use cases and the actors interacting with the system.

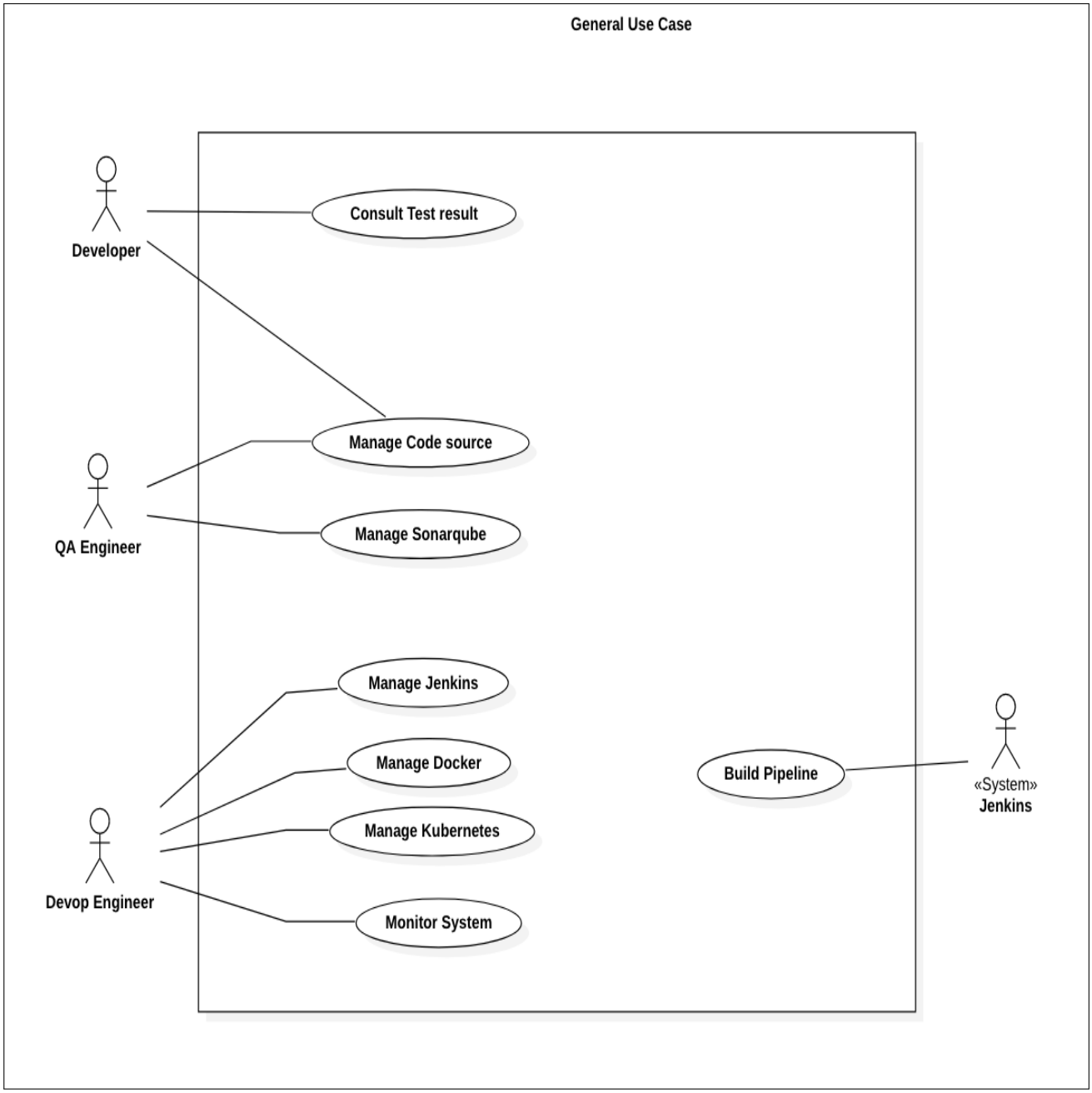


Figure 4.1: General use case of the project

To better understand our use cases, they must be explained one by one.

* **Manage Code Source:** Both the developer and the QA tester can get the Clone the repository and modify it.

* **Consult test result:** The Developer can consult the test result report (Unit Integration) sent by email from Jenkins.

* **Manage Sonarqube:** Sonarqube is managed by the QA engineer in order to assess the accuracy of the source code and review the metrics.

* **Manage Jenkins:** DeVops engineer is in charge of managing the creation and deletion of Jenkins projects, as well as managing users and their access privileges and Jenkins configuration.

* **Manage DeVops:** Creating docker image and manage the containers with docker compose which is managed by the DeVops engineer.

* **Manage Kubernetes:** The DeVops engineer configure a Kubernetes cluster hosting our containers that contains our applications and manage it with google cloud platform.

* **Build Job:** Jenkins' task is to start a build that will run the pipeline.

4.4.2 BPMN diagram of the global process

To completely comprehend our system's functional behavior, we created a general BPMN diagram, as seen in Figure below. This diagram shows the proposed system's overall process.

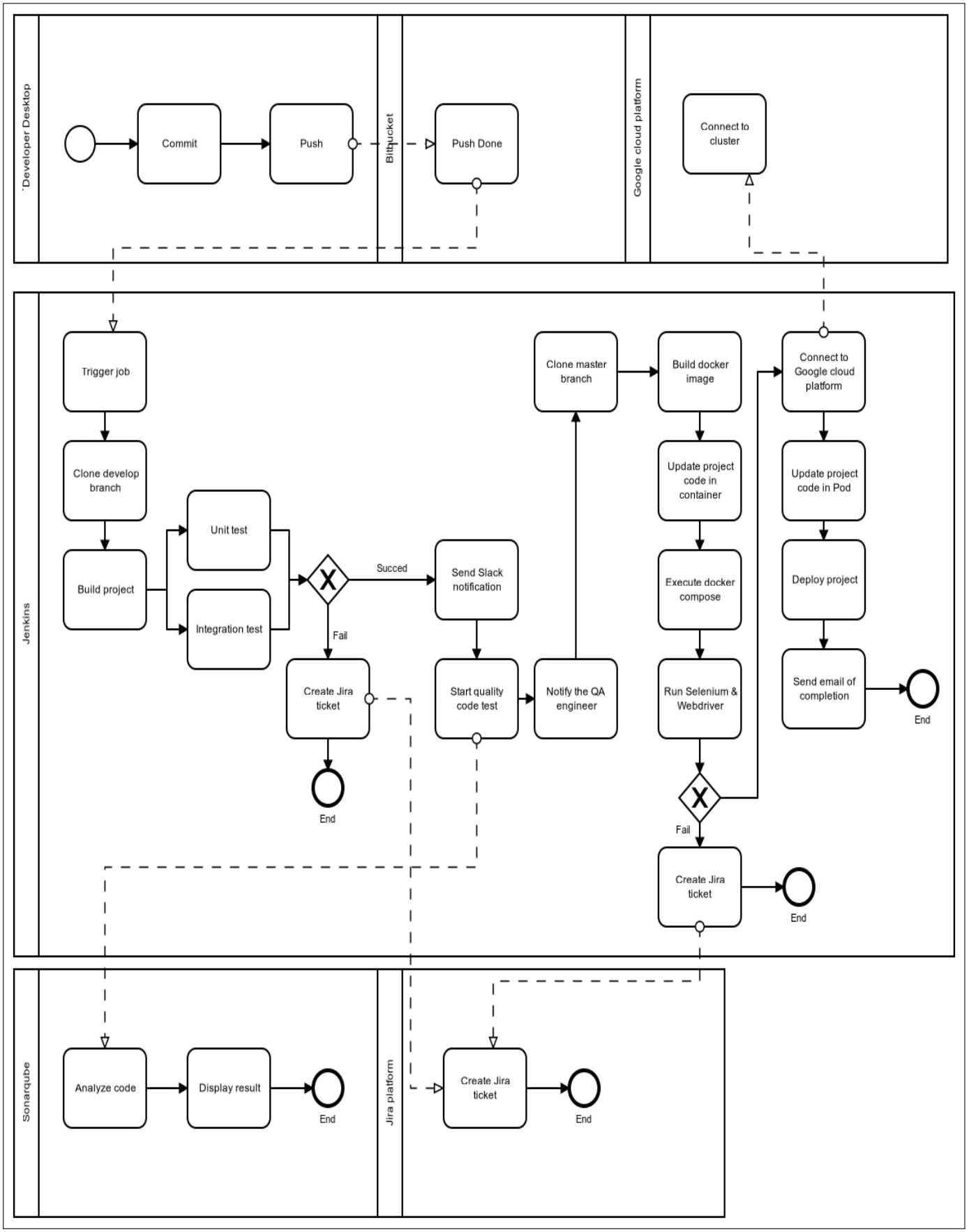


Figure 4.2: BPMN of the project

In this diagram, we see a Developer who is in charge of integrating new functionalities and Implementing their test cases. We also find the different tools chosen, which are Jenkins as a CI server, Bitbucket for source code management, Sonarqube for software quality inspection, and Google Cloud as our cloud provider, as stated in chapter three "comparison of DeVops tools". The following are the main stages in this procedure:

① **Before the process:** The developer clones the code into his private workspace. He then makes the necessary changes and tests them locally.

② **Commit push:** Once finished, the developer archives his changes in Bitbucket.

③ **Trigger job:** Jenkins constantly tracks the source code repository and executes the job when it detects an update.

④ **Clone develop branch:**When the pipeline starts, it will clone the develop branch where our project code reside.

⑤ **Build project:** Jenkins will build our project by a set of commands.

⑥ **Test project(unit and integration):** The process will launch both tests if one of the tests fails it will create A Jira ticket , inform the developer and finally will end the process. If it succeeds, the process will move on to sending a slack notification to the developer.

⑦ **Start Quality code Test**: In this step, Jenkins will launch a request to Sonarqube to analyse our code and display the result.Afterward the QA engineer will be notified By slack.

⑧ **Clone master branch:**In this step , we will clone another branch "master" that contains all the configuration needed to create docker images and manage our containers.

⑨ **Build Docker image:** Using Docker, we will create an image and build it to obtain our container.

⑩ **Update project code in the container:**Pull the latest version of the project in the container that contain our application.

⑪ **Execute Docker Compose:**Executing all the containers that contain the components we will be using for our project and connecting them using docker-compose.

⑫ **Run selenium and Webdriver:**Running both Selenium and a webdriver,in our case chromedriver, to prepare for the functional test.

⑬ **Run functional test:**Similarly to the unit and integration test , our application will run the generated functional test. If it fails it will create a Jira ticket and inform the developer and end the process and if it succeed it will move to the next step.

⑭ **Connect to google cloud:**To be capable of using our Kubernetes features we will need to connect to our cloud provider , google cloud platform which will connect us to our cluster.

⑮ **Update project in Kubernetes:**Since pods in our Kubernetes cluster are the units that contains our containers , we need to update it with the new update.

⑯ **Deploy project:**For our project to be deployed, we need to run a set of commands to deploy out project.

⑰ **Send Notification:**when all the stages are successful in our Jenkins pipeline we will inform our DeVops engineer.

4.5 Detailed conception model

The purpose of this work is to provide a generic CI/CD method for every kind of project. To do this, we have designed use case diagram and a global BPMN diagram. However, in order to bring the proposed process into effect, we must first decide the tools that will be used to develop the software under consideration.

4.5.1 Tools used by the company

A thorough search process is needed when selecting an e-commerce platform. A company will consider what they can do with a platform and how the platform's capabilities can fulfill their requirements. Softifi uses, for most of its projects, the Magento platform for their e-commerce websites. This platform is a module-based system that needs a different component to function, which mainly are Nginx as a web server, Mysql as a database, and Elasticsearch as the search engine.

4.5.2 Environment Workflow

Product development teams must maintain an optimized workflow in today's modern business world if they hope to stay successful in a competitive industry. A well-designed workflow not

only makes a team active, but it also aids in the delivery of stable and timely applications[21]. On of the most used setup is development, staging and production environments :

- **Development:** The first line of protection against vulnerabilities will be the development environment. Developers test any newly added functionality and deploy their code here.Also in our case, it's where Quality Assurance is carried out.Before re-deploying for more testing, any bugs discovered are fixed. The method is repeated until the code is ready to move on to the next level of testing.
- **Staging:** When developers are satisfied with their code and believe it to be reasonably stable, it is deployed to the staging area for further testing.Testers gain access to the staging servers and verify that the program functions properly. They run test cases to discover bugs and consistency checks and see if they can make improvements. The developers are notified of any issues or changes, and the process is repeated until the code passes the staging point.
- **Production:** The code is then moved to production and made available to end-users after it has been thoroughly tested.

In our next step, We will indulge more in how we divided our workflow and visualize it in a prototype diagram.

4.5.2.1 Development prototype diagram

In Figure 4.3, we will analyse the first part of our workflow: Development.

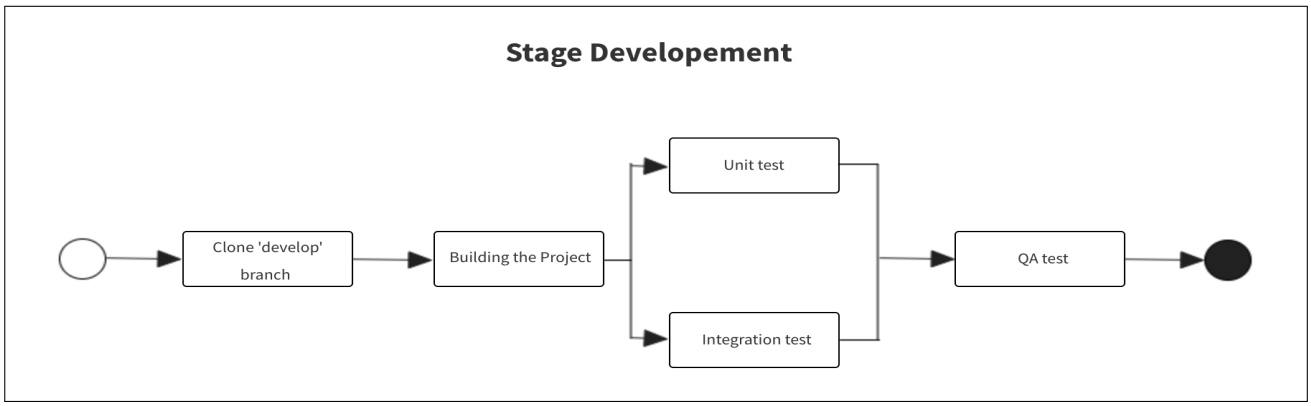


Figure 4.3: Prototype of Stage Developement

Jenkins starts by running the clone code from the develop branch to get the most recent version of the application code from Bitbucket. Afterward, the process will start the 'build project' step where it upgrades the project modules, compile the code, clean the previous cache and

finally build it and deploy it in the development environment, which is Localhost. Then the unit and integration will start parallelly. Those tests are premade from Magento, and some are made by developers for their custom modules. And Lastly, we finish this stage with a Quality code test to ensure the good quality of our code.

4.5.2.2 Staging prototype diagram

We will be discussing the second step of our workflow Staging in the figure 4.4 below. This

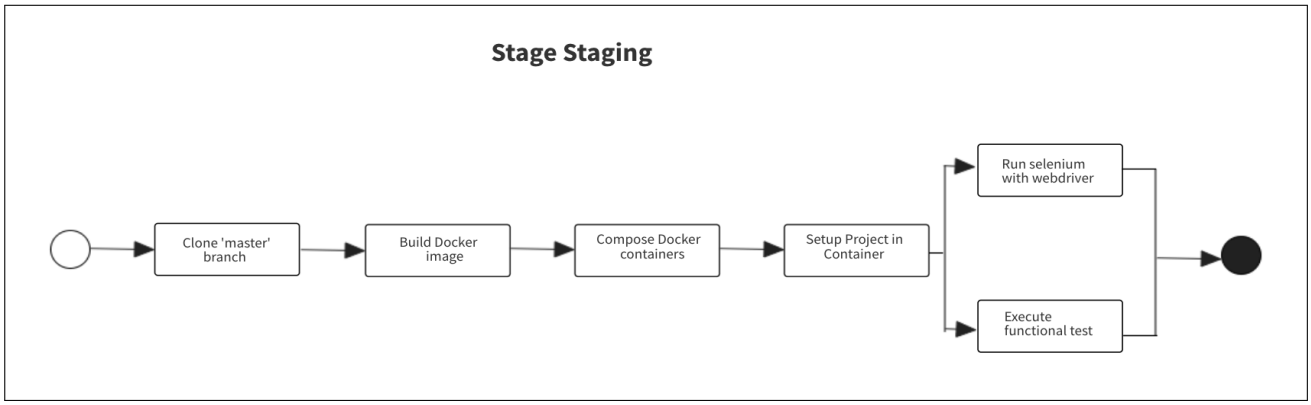


Figure 4.4: Prototype of Stage Staging

process is launched after we made sure that all the "white box" tests are successful and the quality of our code is promising. In this stage, we will start with cloning the "master" branch, that contains our docker configuration and a sub-module that contains the "develop" branch repository. Building our docker image will be our next step. This image is configured to host our application by lessening our CPU resources. Afterward, using docker-compose, we will ensure that all the needed containers will run accordingly to ensure that our Magento application is running. To test if the application is running, we will run functional tests using selenium and a web driver, which is chromedriver in case.

4.5.2.3 Prototype of Stage Production

The steps that make up the stage Production are illustrated in the following figure 4.5. After ensuring that the new update in the application is safe and functioning, we move to our final stage: Production. In this process, we will start by connecting to our cloud provider, the Google cloud platform, which hosts our Kubernetes cluster. And since our application is functioning in this in a pod, unit in Kubernetes, we will need to pull the latest update from Bitbucket and finally deploy it to users.

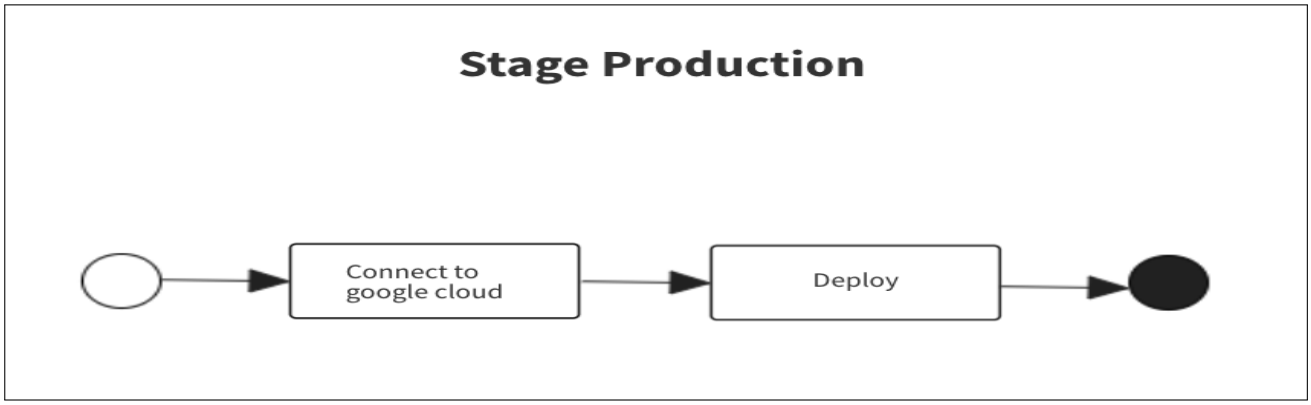


Figure 4.5: Prototype of Stage Production

4.5.3 Kubernetes Architecture

In this section we will be explaining the architecture of our Kubernetes cluster and how we deployed with the help of the figure 4.6 bellow.

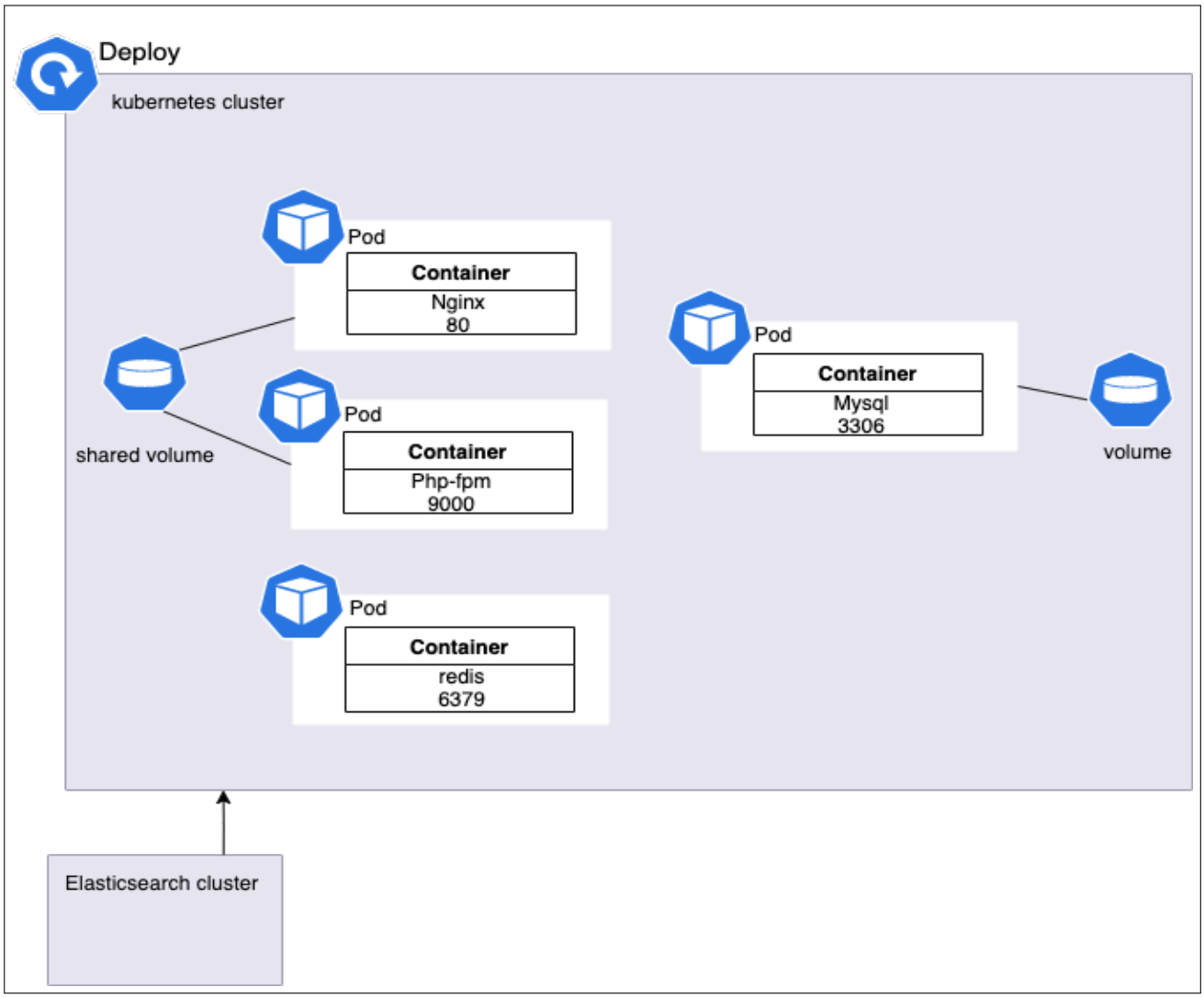


Figure 4.6: Overview of Kubernetes cluster

To be able to use a Kubernetes cluster, we need a cloud provider to create it manage it and in our project, we got the chance to work with the Google cloud platform and using the google Kubernetes engine as the API. As mentioned previously, to run a Magento project, we need Php, Nginx, Mysql, and Elasticsearch. Therefore we created a cluster that contains Nginx pod as our web server and a PHP-fpm pod that contains the needed configuration to host our project. Both pods share the same volume since Nginx needs to reach our project for configuration purposes. Also we have mysql pod with it's volume. finally we have Redis pods to manage our project. this pods where created with replicas to ensure high availability. and since elastic search has a pre-made cluster we skipped creating a pod. But for pods to interact with each other, it needs services as shown in the figure 4.7

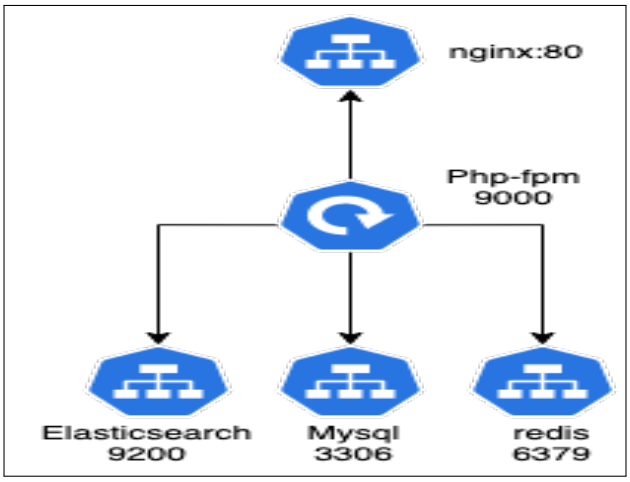


Figure 4.7: Interaction of Kubernetes services

We have our Magento project in PHP-fpm using MySQL, elastic search, and Redis by their endpoint ports and having an Nginx service using a load balancer to expose our project to the internet.

4.5.4 Refined use case Diagram

All of the DeVops engineer's responsibilities are mentioned in this use case diagram:

- **Manage Jenkins:** This task includes three main responsibilities which are : Creating Jenkins job such as pipeline job and Freestyle Jobs. Also installing needed plugins for the jobs.Finally configuring Jenkins to fit the requirements.
- **Manage Docker:** Managing docker involves you to create docker images using YAML configuration files, executing containers with docker-compose and building docker files.

- **Manage Kubernetes:** Kubernetes management entails creating the needed pods and services and managing a cluster to host the pods and exposing the services.
- **Monitor Kubernetes cluster:** Using grafana and prometheus, DeVops engineer will analysis our cluster resources uses.

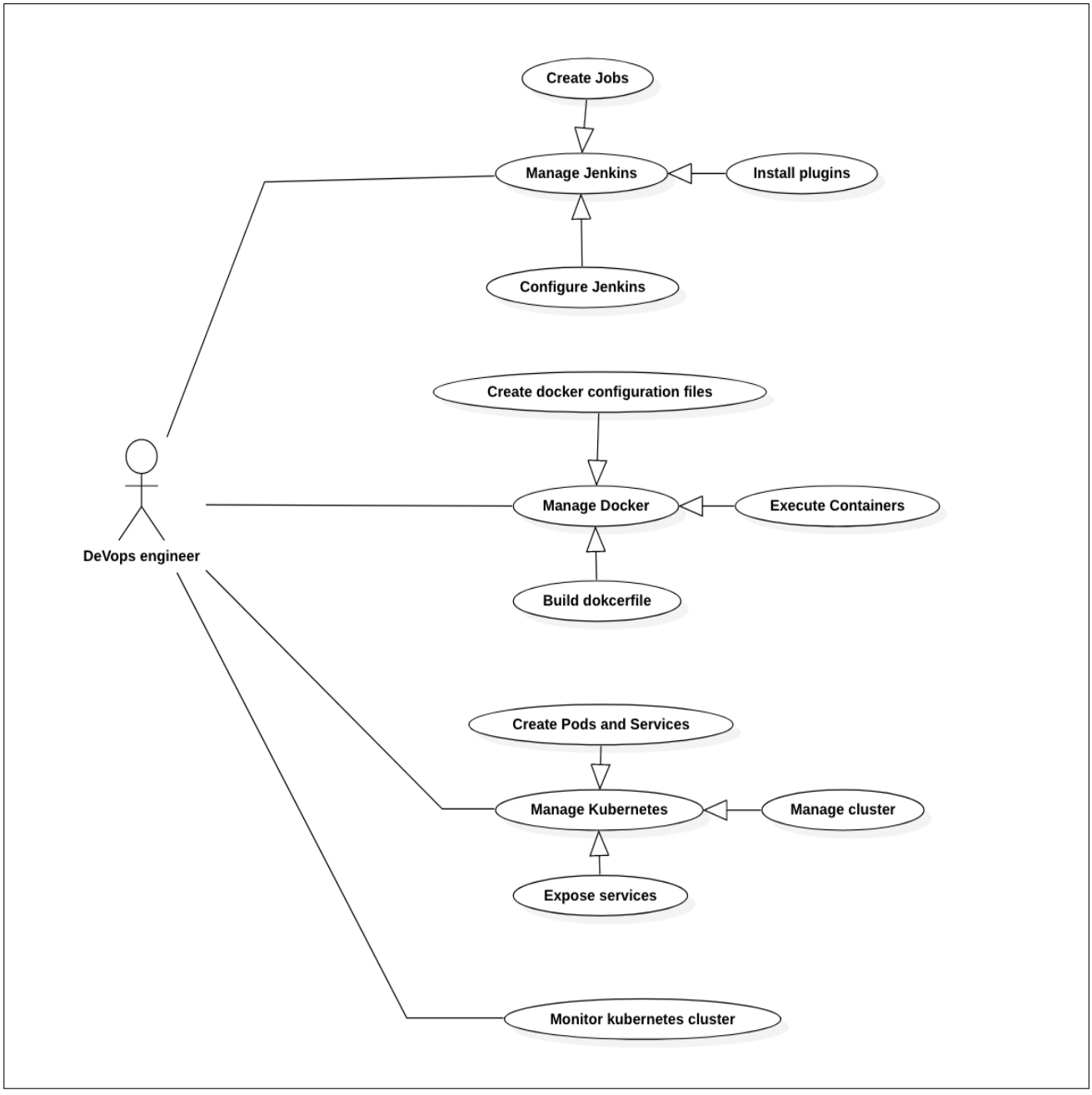


Figure 4.8: Refined use case Diagram

4.5.5 Sequence Diagram

The **build pipeline** in the general use case is illustrated in the sequence diagram in the figure 4.9.

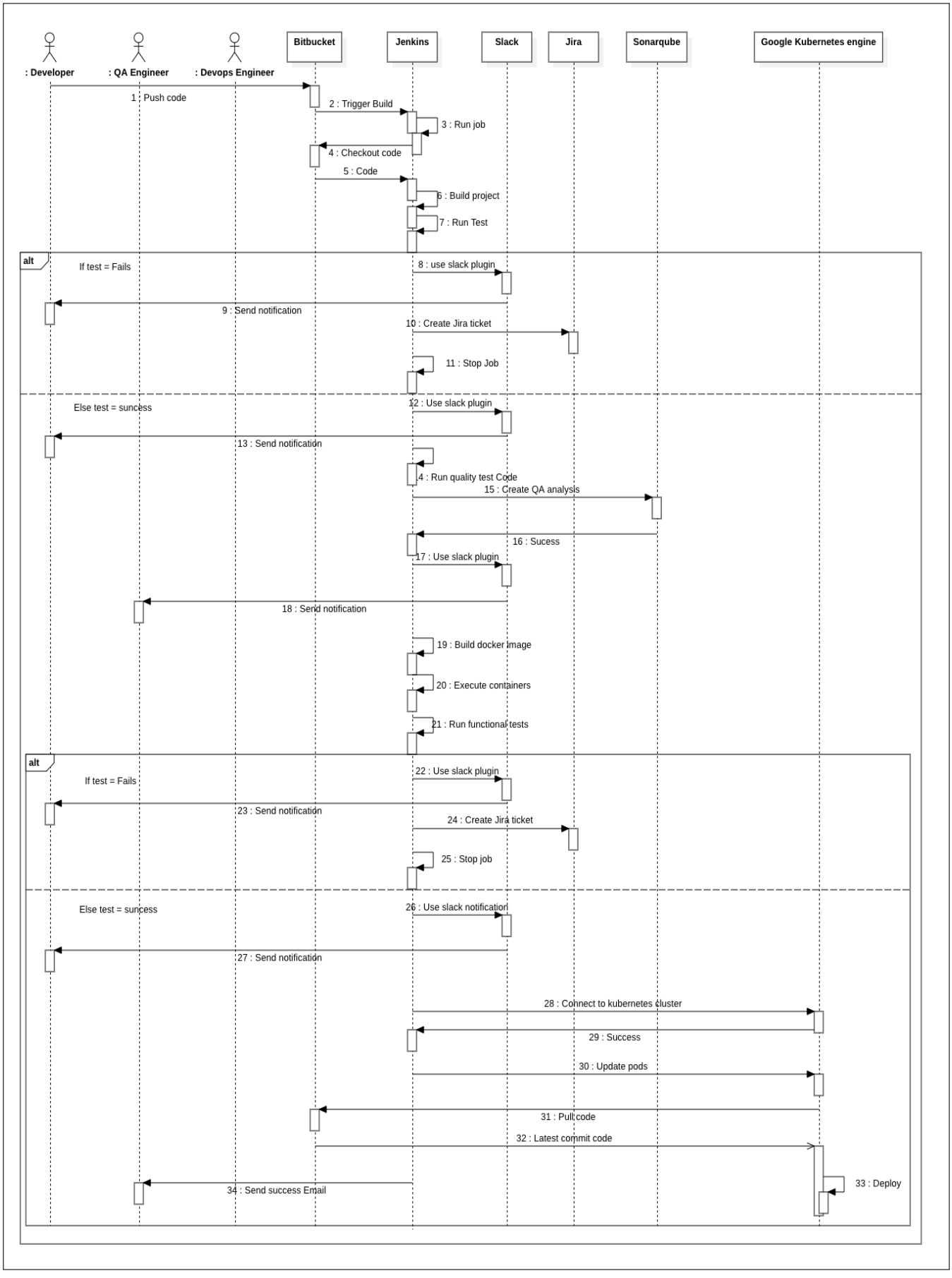


Figure 4.9: Sequence Diagram for "Build pipeline"

4.5.6 Componenet Diagram

The components that make up our CI/CD structure are shown in figure 4.10 below. The three Jenkins components, Sonarqube, Jira, and Bitbucket, must all be started at the same time on different ports at first. Jenkins then begins the pipeline execution once it detects that a developer has recorded new code in Bitbucket.

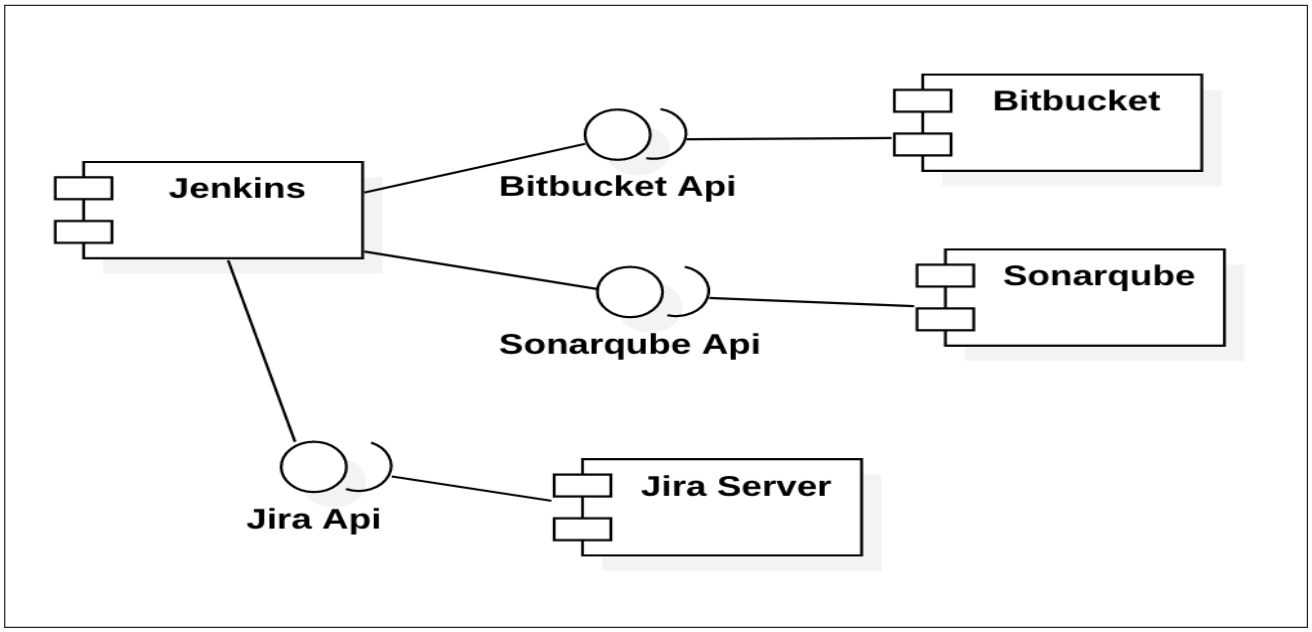


Figure 4.10: Component Diagram

The Jenkins integration server will interact with the other components, as seen in the diagram above. It uses the Bitbucket interface to get the code, Sonarqube interface to get the reports, and the Jira interface to manage issues.

4.5.7 Deployment Diagram

The deployment diagram shown in figure 4.11 was designed to represent the physical architecture of our system. This diagram portrays the system's physical nodes and their interactions.

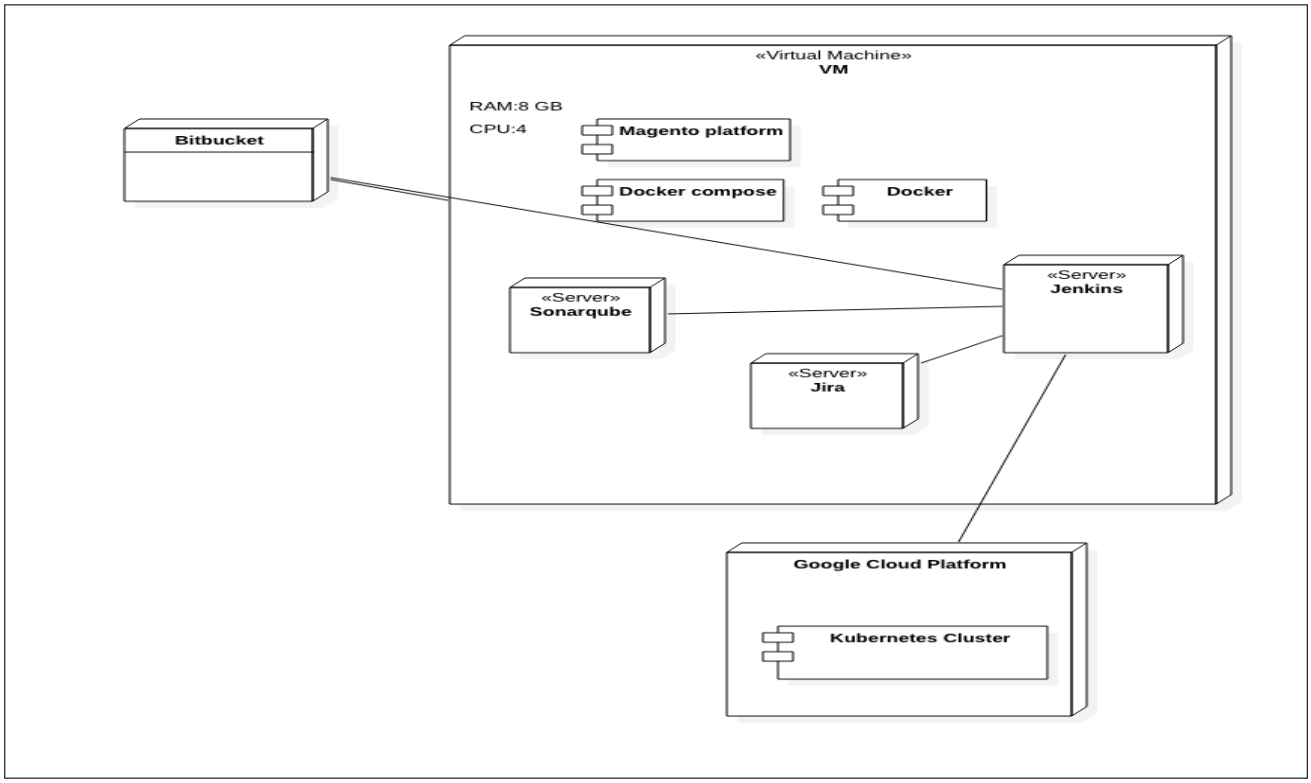


Figure 4.11: Deployement Diagram

Setting up a CI/CD system that can accommodate up to 500 users requires the use of capable machines. We install the Jenkins CI server, the utilities expected for our project, Sonarqube and Jira, and the two Docker components: Docker and Docker compose on the computer with 8 GB RAM.

Conclusion

The functional and non-functional specifications of our system have been addressed in this chapter. The BPMN diagram, use case diagram, component diagram, and deployment diagram are all diagrams that illustrate the configuration of our solution. This stage serves as a warm-up for the next chapter, "Implementation and Realization."

Chapter 5

Implementation And Realization

Plan

1 **Sprint 1 : Preparing the working environment** **51**

2 **Sprint 2 : Creating a Jenkins pipeline job** **54**

3 **Sprint 3 : Preparing tests for the project** **58**

4 **Sprint 4 : Installing and integrating Sonarqube** **59**

5 **Sprint 5 : Dockerazing the project** **60**

6 **Sprint 6 : Deploying the project using Kubernetes** **61**

7 **Sprint 7 : Monitoring** **62**

8 **Sprint 8 : Integrating all the tasks in the pipeline** **63**

Introduction

We introduced the concepts required for a thorough understanding of our project in the previous chapters, which were based on a preliminary study that helped us to put the project in its overall context, as well as a comparative study and a comprehensive design of our solution to be implemented.This is the final chapter of the project, and it will explain the various steps involved in implementing our final solution. Second, we'll go into the hardware and software environment, as well as the various technologies used.The steps taken to arrive at the final result will then be presented. Finally, using screenshots, we will explain the key functionalities of the implemented solution.We will be dividing them with sprints we mentioned in the first chapter.

5.1 Sprint 1 : Preparing the working environment

With the aim to set up our working environment, we used very specific hardware and software resources to complete this project, as we will explain in the following sections.

5.1.1 Hardware

To carry out our project, we used as hardware environment, a virtual machine with the following characteristics:

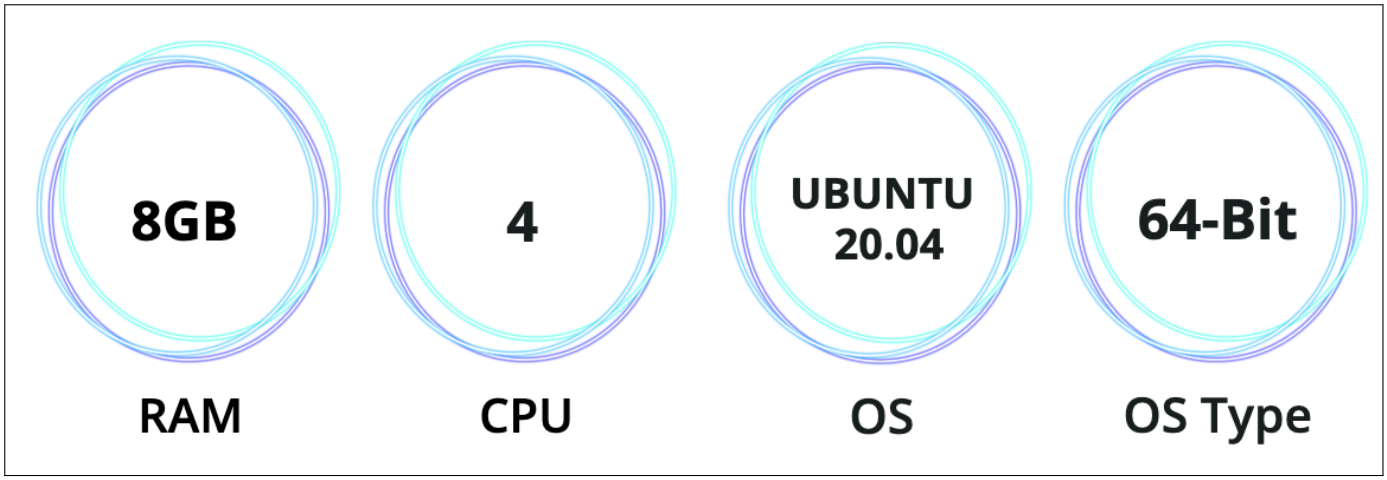


Figure 5.1: VM specifications

5.1.2 Presentation of the Magento project

After installing Magento in our machine, we prepared a prototype project to apply the solutions planned in chapter four, "Analysis and conception". Our prototype consists of an e-commerce website that sells electronics goodies called "Tajer Pro". In Figure 5.2, we display the application's Home interface.

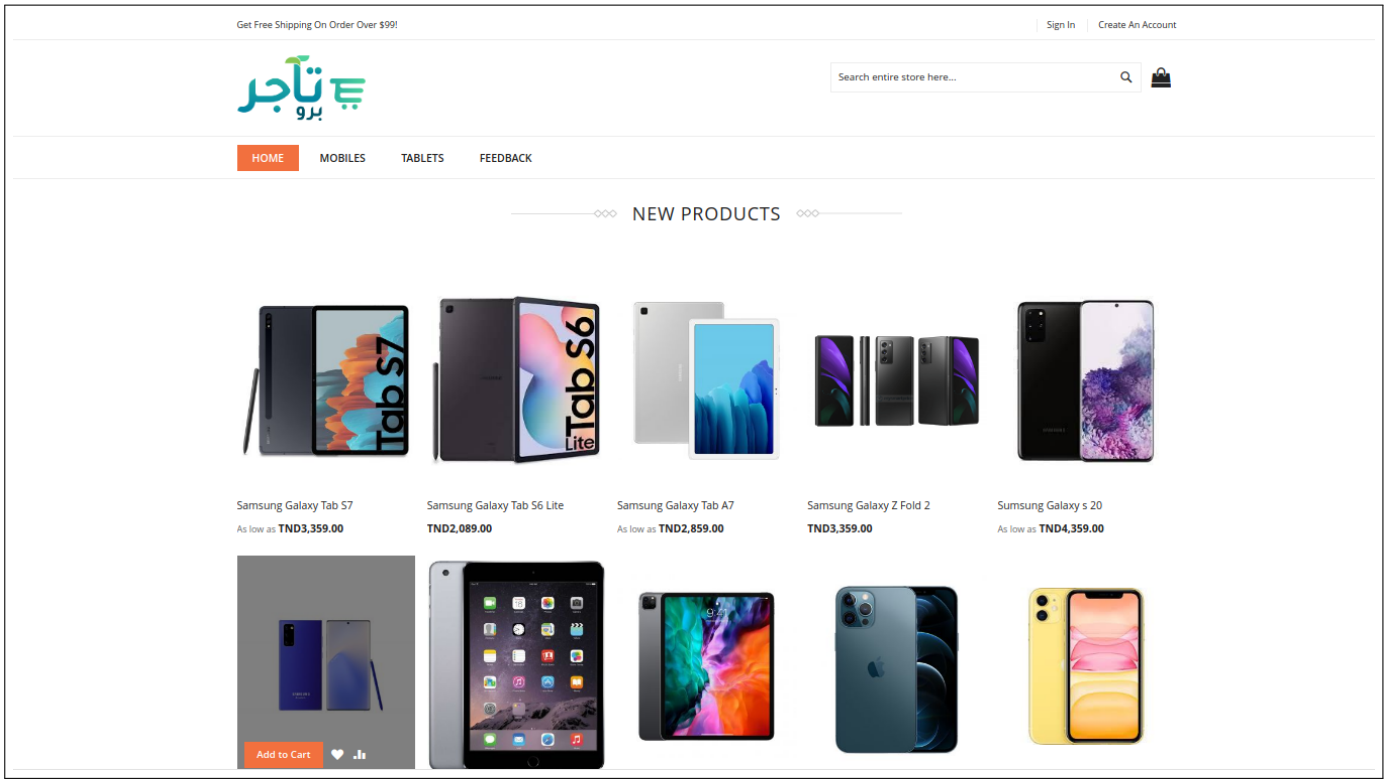


Figure 5.2: Tajer pro home interface

5.1.3 DeVops Tools used

Several softwares were used during this project, which we can divide into two groups. The first group is set up to test and execute projects until they are integrated with Jenkins. The suggested solution is implemented in the tools used for pipeline implementation mentioned in the next page.

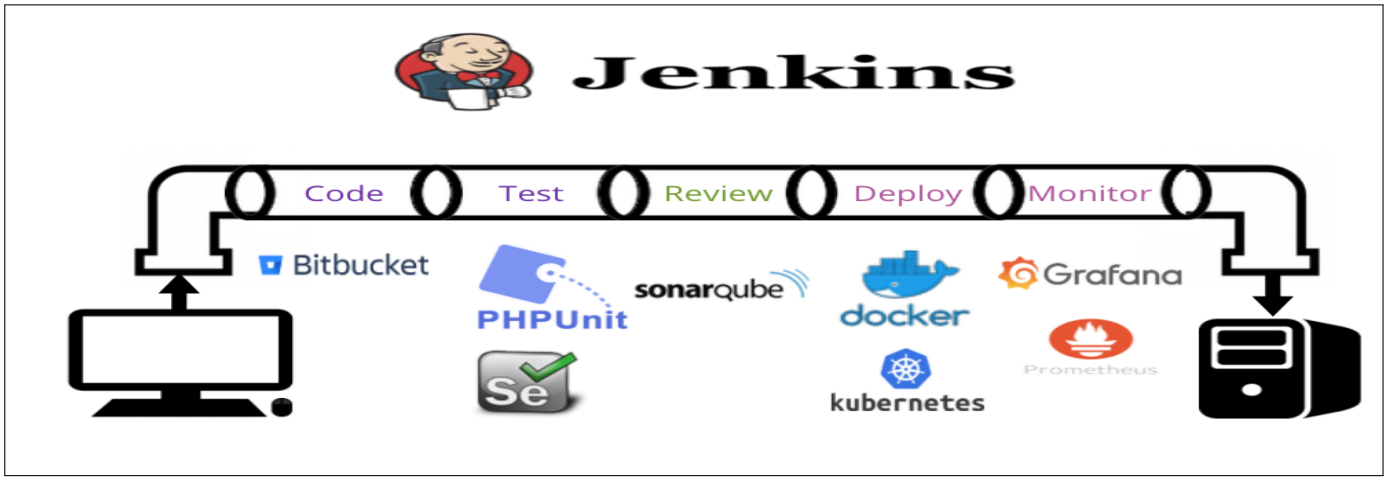


Figure 5.3: Tools used in the DeVops workflow

- **Tools used for project development:**

  - **Platform:** Magento
  - **Web server:** Nginx
  - **Database** Mysql
  - **Programming Language:** PHP
  - **Search Engine:** Elasticsearch
  - **Package manager:** Composer
  - **Test framework:** PHPUnit
  - **Functional Test Tool:** Selenium

- **Tools used for pipeline implementation:**

  - **CI Server:** Jenkins.
  - **Version Controller:** Git.
  - **Code source manager:** Bitbucket.
  - **Quality Code Tool:** Sonarqube.
  - **Container-based virtualization platform:** Docker.
  - **Container Manager:** Docker Compose.
  - **Orchestrator:** Kubernetes.

- **Tool used for System monitoring:** Grafana and prometheus

5.2 Sprint 2 : Creating a Jenkins pipeline job

Using Jenkins jobs requires first installing the server in our machine and setting it up with different plugins to finally be able to create a Jenkins job which will be covered in this section.

5.2.1 Jenkins Installations

To install our CI server Jenkins in our machine, we appended our Jenkins deb file in the system sources and used the known apt-get install to install our service. We used the default port 8080 and created an admin user to access the platform.



Figure 5.4: Login interface in Jenkins

5.2.2 Jenkins Global Configuration

For Jenkins to manage our application, we had to install a couple of plugin in the Jenkins plugin manager, such as git, Bitbucket, Phpunit, Clover coverage and such.

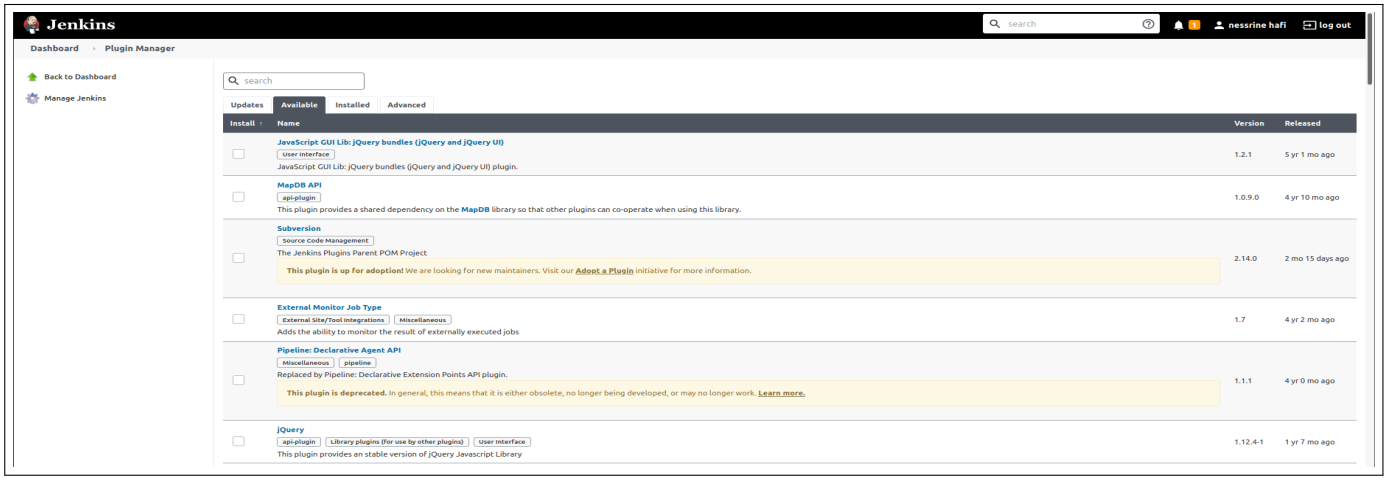


Figure 5.5: Jenkins plugin manager interface

We also configured the SMTP settings in Jenkins management interface to receive email notifications from Jenkins. We created a Gmail account to send notification and we used SSL as our protocol using port 465.

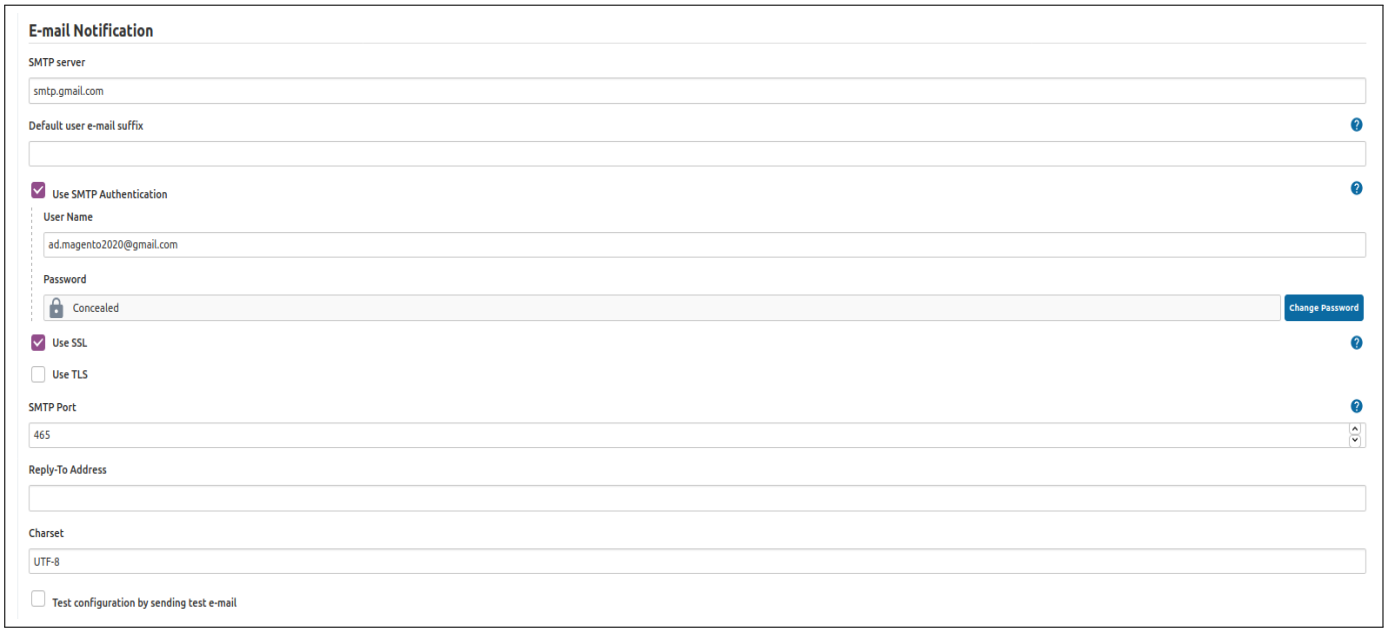


Figure 5.6: Email notification setting interface

Also to notify the employees, we configured our slack plugin to be linked with the designated workspace as shown bellow.



Figure 5.7: Slack plugin setting interface

5.2.3 Plugin configurations

We built a series of configurations to allow the installed components to interact with one another. Since Jenkins is our system's arranger, we started there. We then combined it with

other tools such as Bitbucket and Jira, among others.

- **Jenkins with Jira:**

  To be able to use Jira in Jenkins, you need to install it on your machine. Creating a PostgreSQL database to store the data and assign a user for Jira will be the first step. After, we will download the deb file that contains the needed file to install Jira, which will be hosted at our 8555 port on our device since the default port is used by another service.

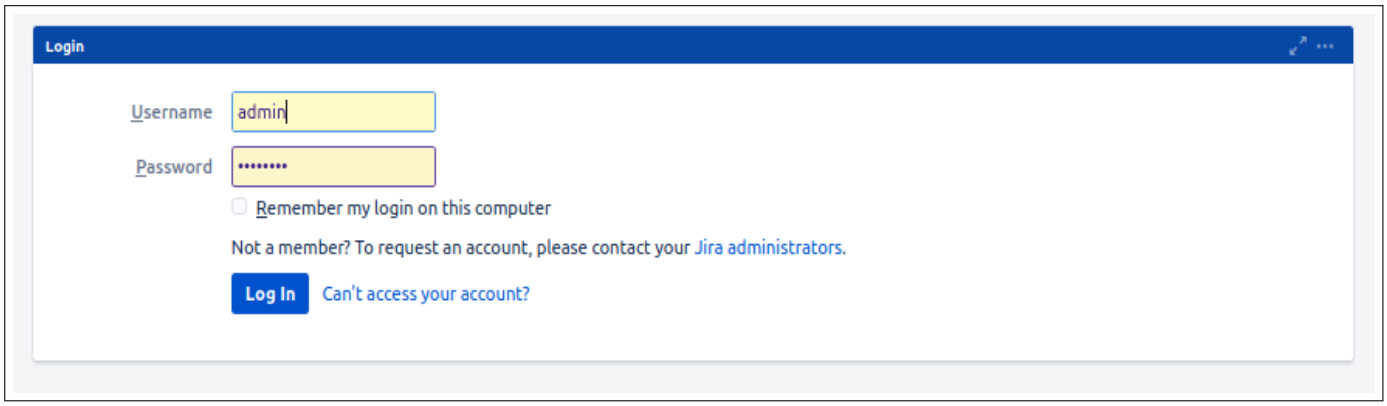


Figure 5.8: Jira login interface

Afterward we headed to our configuration interface to set up Jira steps by adding a name that we will use in our pipeline in the following steps, the servers url , the type of login we will use to connect to Jira which in our case is basic.



Figure 5.9: Jira server setting interface

- **Jenkins with Bitbucket:** We first implemented Bitbucket access parameters to Jenkins in order for it to obtain the source code. The webhook was then programmed to alert Jenkins when new code was introduced by Bitbucket.

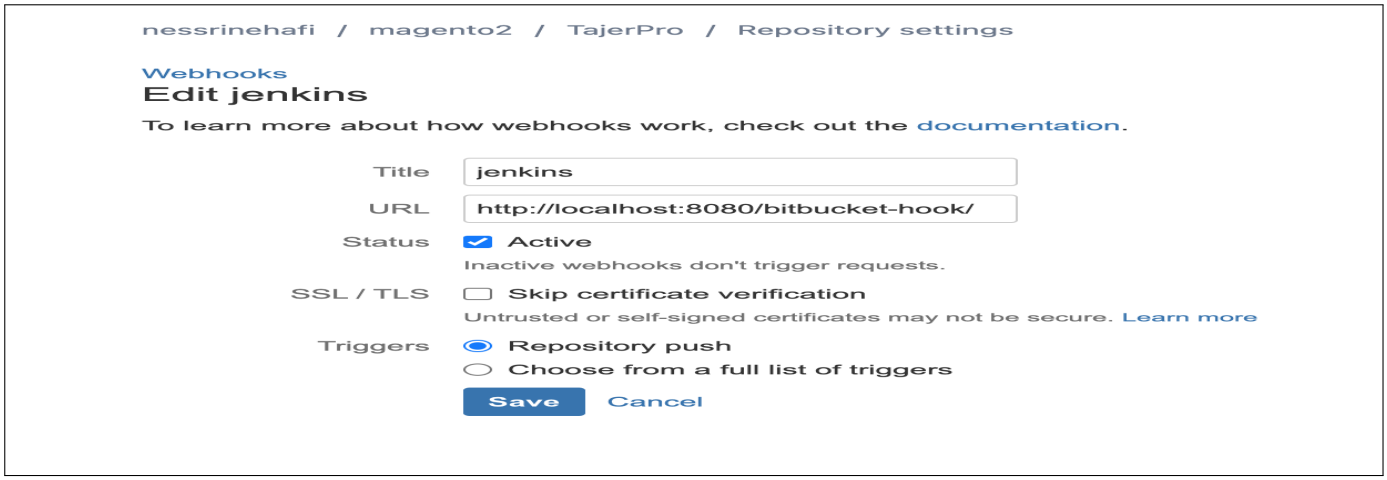


Figure 5.10: Bitbucket webhook setting interface

5.2.4 Jenkinsfile

Jenkinsfile is a text file that contains the Jenkins pipeline specification, is archived in the source code manager. In our case, we created a Jenkinsfile file that contains all of the steps that Jenkins would perform. This file will be used in the planned Jenkins projects, which we will be detailed in sprint 8 in this chapter.

5.2.5 Creation of Jenkins projects

We can choose between two types of projects while dealing with Jenkins: Freestyle and Pipeline. Jenkins' default feature when making a new project is the Freestyle project. The other is a recent addition with a new feature. Let's take a look at the distinctions between the two.

5.2.5.1 Freestyle Job

This is a general-purpose project for orchestrating basic tasks for a project, such as running tests, generating or packaging an application, or running commands. This kind of project was used in our case to simplify the execution of selenium and our webdriver for functional testing. We built in our project one Freestyle task, as seen in Figure 5.11.

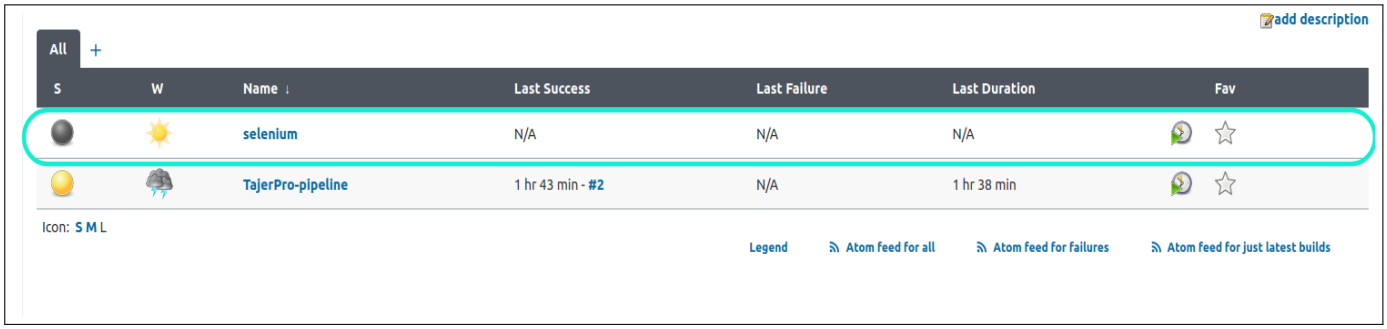


Figure 5.11: Freestyle job 'Selenium' used in our project

5.2.5.2 Pipeline

When we try to set up a "pipeline as a code," we use this new kind of Jenkins project. Users will also execute the entire pipeline in a script file (Jenkinsfile) and store it alongside their code, handling their pipeline as if it were another piece of code, for this style of project. In the case of our work, we created a single Jenkins pipeline task, as seen in Figure 5.12.

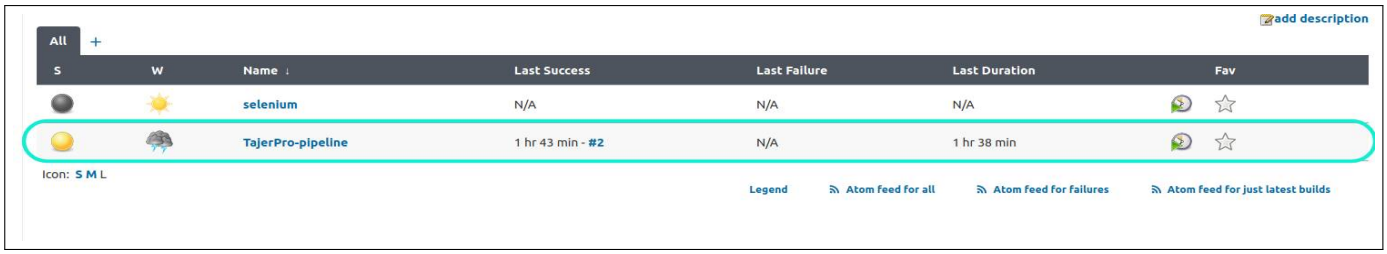


Figure 5.12: Pipeline job 'Tajer Pro' used in our project

5.2.5.3 Final Solution

We merged the two types of jobs to provide the final solution. We did, in fact, build a project pipeline, and within that pipeline, we referred to our Freestyle jobs as being in a specific stage.

5.3 Sprint 3 : Preparing tests for the project

In this sprint, we dived more into testing our project. To do so, we built a simple custom module that exhibits a form to add feedbacks displayed in figure 5.13. Furthermore, we made created various kinds of tests to integrate with our project in our custom modules.

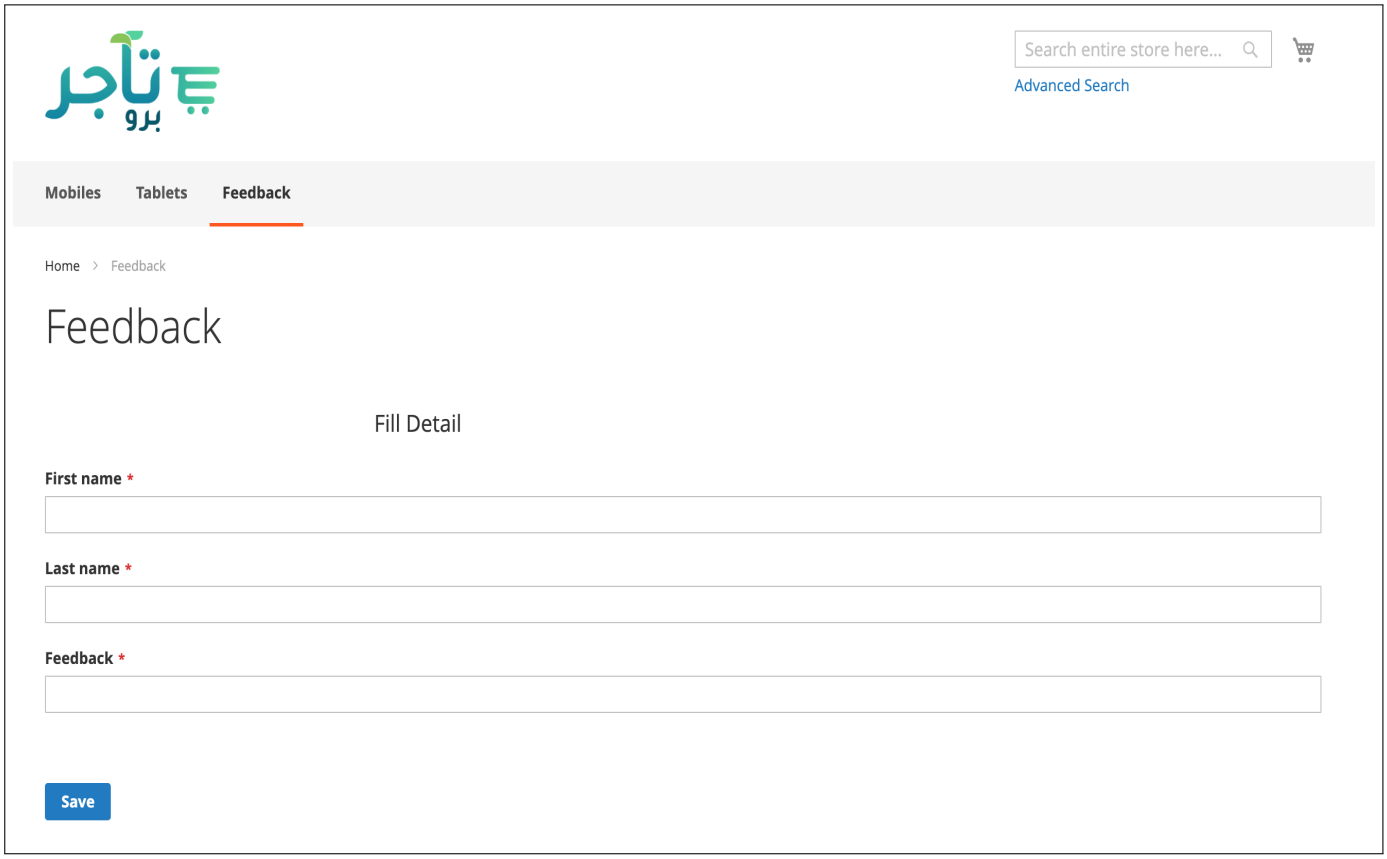


Figure 5.13: Tajer pro feedback interface

5.4 Sprint 4 : Installing and integrating Sonarqube

Similar to Jenkins, we installed our Sonarqube using apt-get but we also installed Sonar scanner that helps Sonarqube to analyze our code. Since this scanner needs a specific java version, we installed it and referenced it in our file configurations. This service use port 9000 and in the figure below is our login page

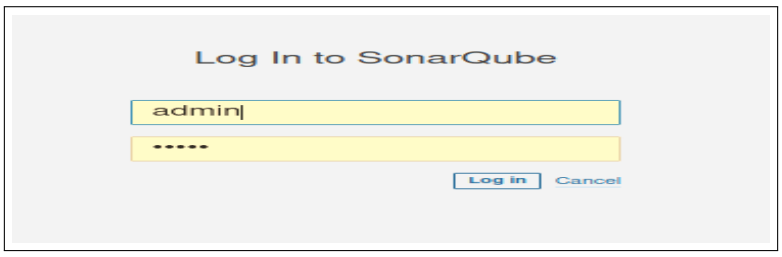


Figure 5.14: Sonarqube login interface

5.4.1 Jenkins with Sonarqube:

We started by creating a token in Sonarqube and setting it up in Jenkins.
Afterward we configure our Sonarqube server by adding the Server URL with their designated port and the token mentioned before.

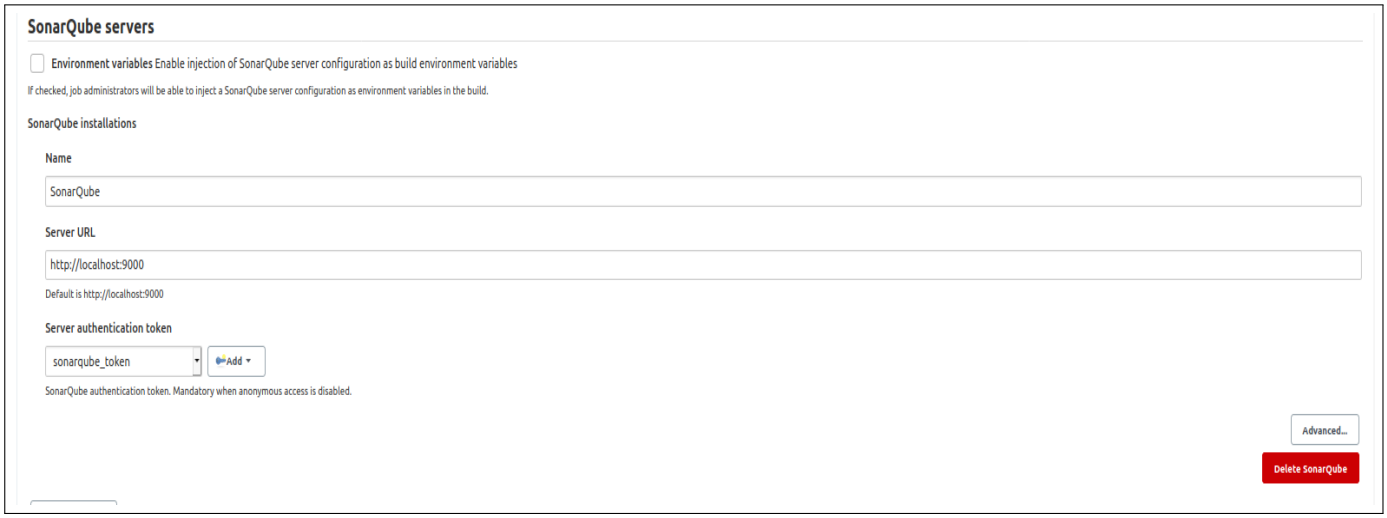


Figure 5.15: Sonarqube plugin setting interface

Since we need our sonar scanner to analyse our code , we use our already installed version in the machine and we set a name for it to use it afterward in our pipeline.

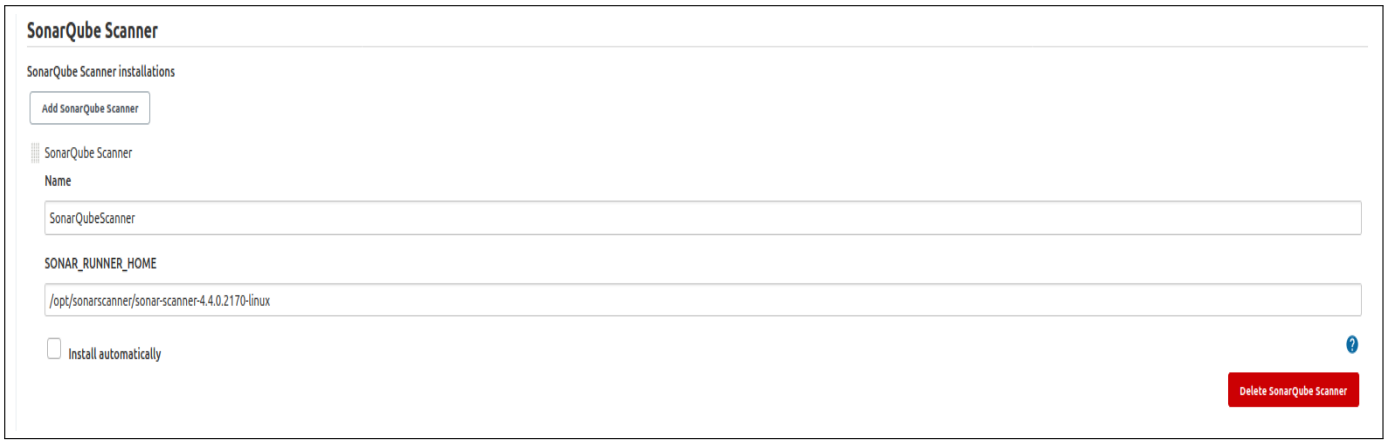


Figure 5.16: Sonarqube scanner setting interface

5.5 Sprint 5 : Dockerazing the project

We prepared the Dockerfile and Docker-compose.yml files so that we could generate Docker images for our project and deploy it in containers. **Appendix A** of this report contains the contents of these files. The commands to be executed to create and the description of all steps to be automated will be illustrated in a Jenkinsfile.

Docker made a big difference in our virtual machine performance. As shown bellow in figure 5.17, the traditional way of installing magento with all the components uses a lot of memory.As a result, our VM performance declines.

```
root@ubuntu:/home/ness# free -h
              total        used        free      shared  buff/cache   available
Mem:          8.5Gi       6.6Gi       1.4Gi        31Mi       530Mi       1.7Gi
Swap:         2.0Gi          0B       2.0Gi
```

Figure 5.17: VM memory before docker

But after switching to docker we notice a shift in the performance of our virtual machine with increased free memory to use.

```
root@ubuntu:/home/ness/tajerpro# free -h
              total        used        free      shared  buff/cache   available
Mem:          8.5Gi       2.3Gi       4.6Gi        64Mi       1.7Gi       5.9Gi
Swap:         2.0Gi          0B       2.0Gi
```

Figure 5.18: VM memory after docker

5.6 Sprint 6 : Deploying the project using Kubernetes

To run our project and containers inside our Kubernetes cluster, we created a set of YAML files that contains different kind of components such as pods , services , deployment , replica-sets and config files. In **Appendix B** we will try to illustrate some of those of files contents. Kubernetes offers different feature to enhance their performance such as High availability and Load Balancing.

- **High availability:**To ensure high availability in our application, we used a deployment controller to create our pods. We commanded in our configuration to create a replica for each pod to avoid single-point failure. In case of a failing pod, there will always be another pod to back up.

```
nessrine_hafi@cloudshell:~/magento2-kubernetes/php-app (pre-project-305009)$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE     IP           NODE                                                NOMINATED NODE   READINESS GATES
elasticsearch-es-elasticsearch-0    1/1     Running   0          8d      10.56.2.8    gke-magento-cluster-default-pool-3608be3d-xu8a      <none>           <none>
mysql-0                             1/1     Running   0          8d      10.56.1.6    gke-magento-cluster-default-pool-3608be3d-41vh      <none>           <none>
nginx-5f9ff476b-g8klv               1/1     Running   0          9m23s   10.56.2.19   gke-magento-cluster-default-pool-3608be3d-xu8a      <none>           <none>
nginx-5f9ff476b-spxm8               1/1     Running   0          9m23s   10.56.2.18   gke-magento-cluster-default-pool-3608be3d-xu8a      <none>           <none>
php-864f67644d-g7vjb                1/1     Running   0          6m58s   10.56.2.20   gke-magento-cluster-default-pool-3608be3d-xu8a      <none>           <none>
php-864f67644d-v8f4v                1/1     Running   0          8d      10.56.2.7    gke-magento-cluster-default-pool-3608be3d-xu8a      <none>           <none>
redis-0                             1/1     Running   0          8d      10.56.1.5    gke-magento-cluster-default-pool-3608be3d-41vh      <none>           <none>
```

Figure 5.19: overview of a set of pods

- **Load balancing:** To expose our application to the internet, we used load balancing as the service for our HTTP server Nginx. Kubernetes load balancer helps you access the service outside of the cluster and makes sure to distribute the load on the available pods.

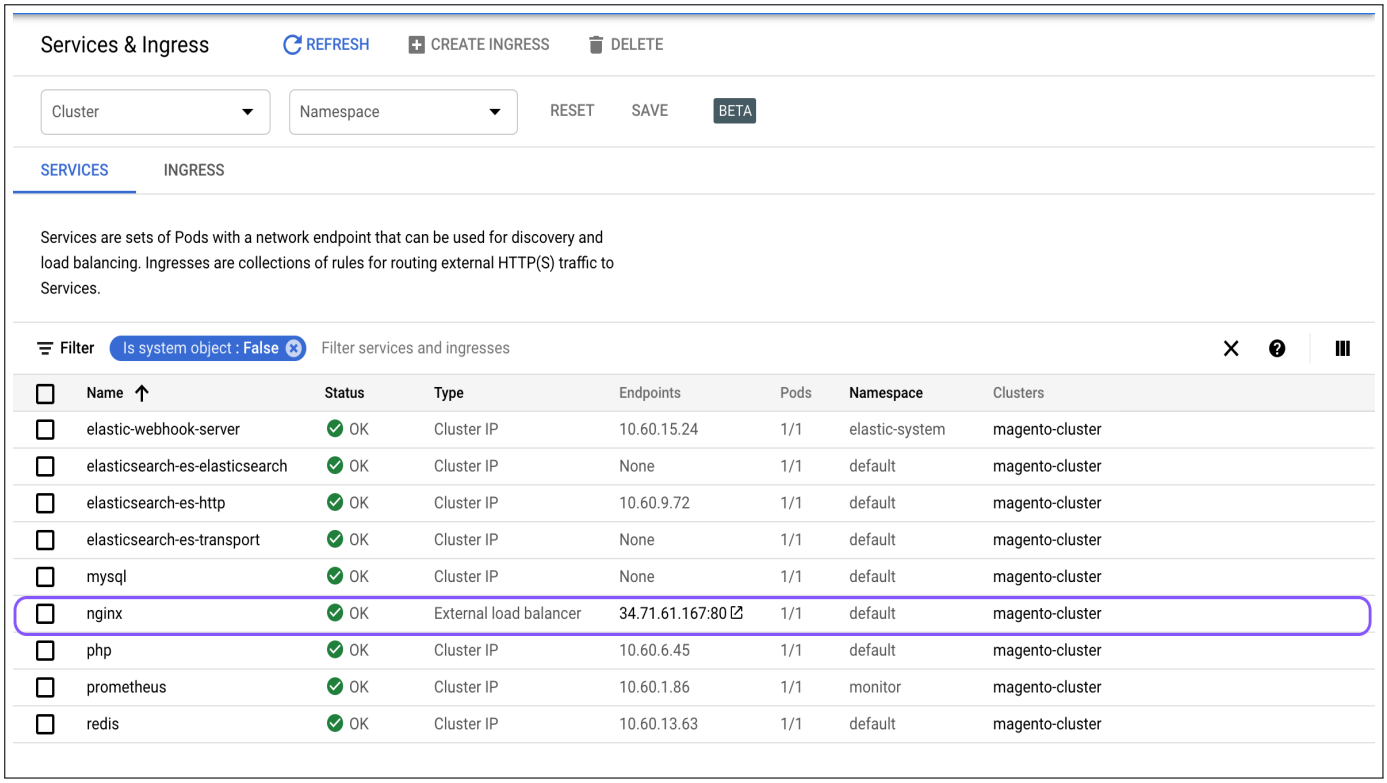


Figure 5.20: Bitbucket webhook setting interface

5.7 Sprint 7 : Monitoring

In order to Monitor our deployment environment, we used grafana and prometheus to visualize how our Kubernetes cluster is using the resources in google cloud. In the figure bellow in the grafana platform displaying a dashboard that contains our cluster resources use such as memory and CPU we also have the same for pods network and so on. and since for grafana to analyse our data it needs a server to store it and in our case prometheus that a plays a back-end role in this monitoring system.



Figure 5.21: grafana interface of the Kubernetes cluster

5.8 Sprint 8 : Integrating all the tasks in the pipeline

We were able to combine several independent components to make them work together in a single process for our project model after a series of installations and configurations.

The solution modeled in Chapter 4 "Analysis and Conception" is implemented in this pipeline.

The following figure 5.22 depicts the general view of this pipeline.

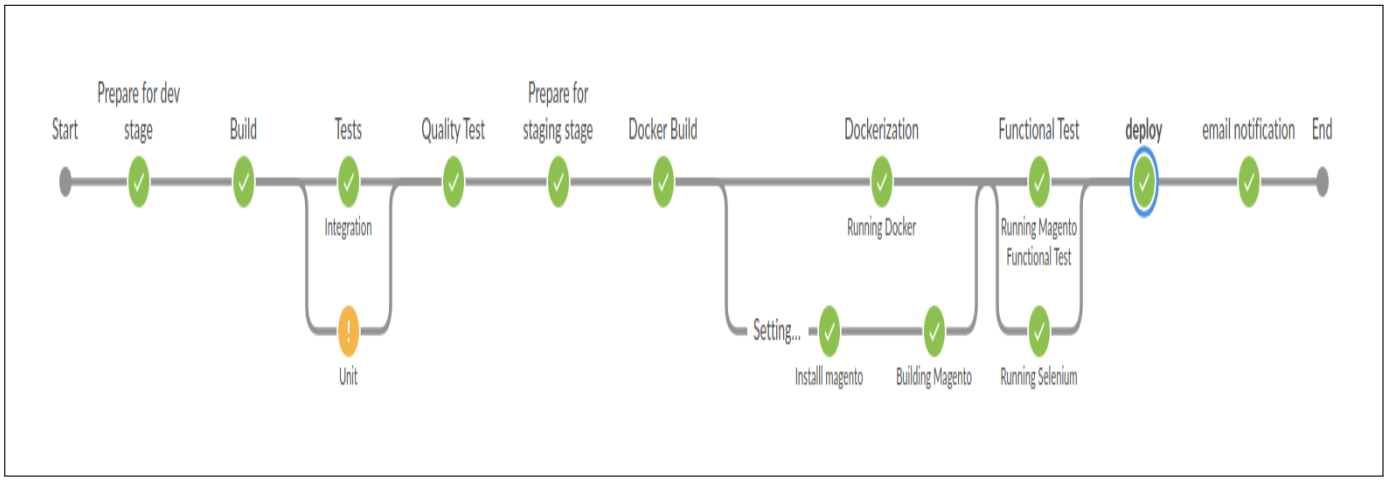


Figure 5.22: Overall view of the completed pipeline

The pipeline is made up of a series of "stages," each of which refers to a set of commands executed by Jenkins. The green color indicates that the step was completed successfully.
The developer must log in to the Jenkins interface using the access information provided by the administrator to see the logs and reports created at each point. Then he must go to the most recent task he made.

- **Test:** This move enables you to run both unit and integration testing at the same time, as well as produce a test result summary that can be accessed through the Jenkins portal. The figure represents a portion of this study.

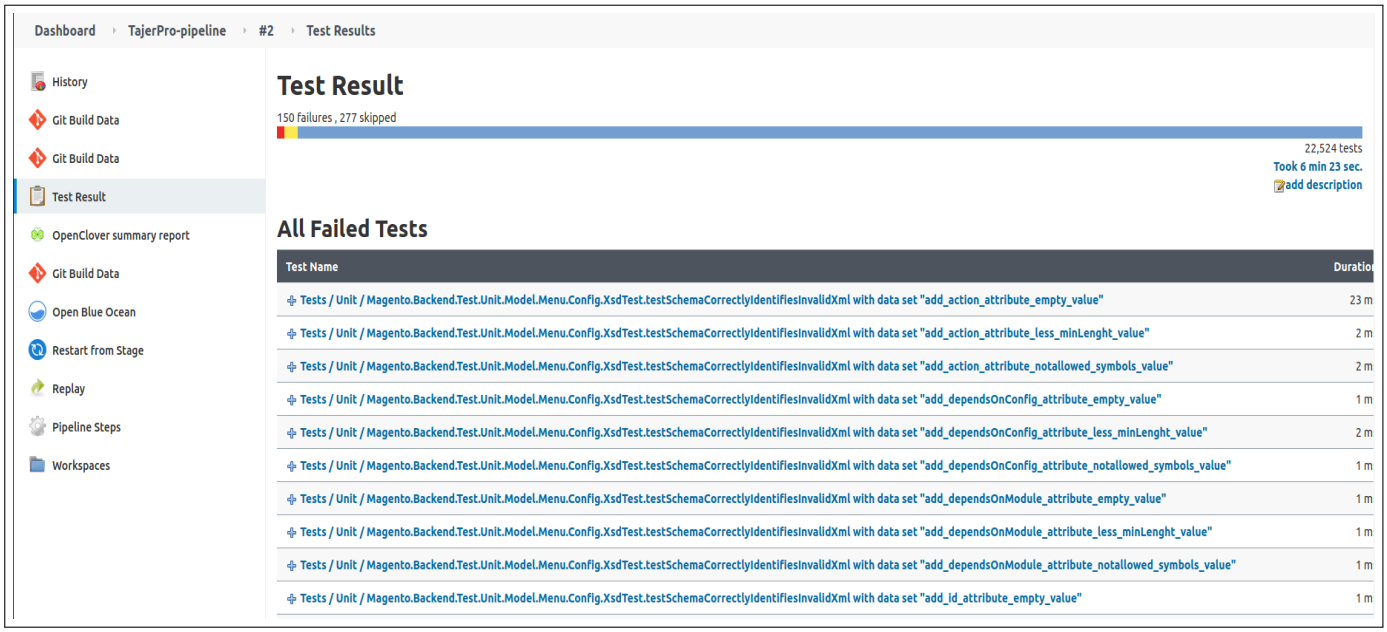


Figure 5.23: Overall view of the completed pipeline

We also have the generated code coverage result done by PhpUnit and Clover , the plugin for coverage ,installed in Jenkins.

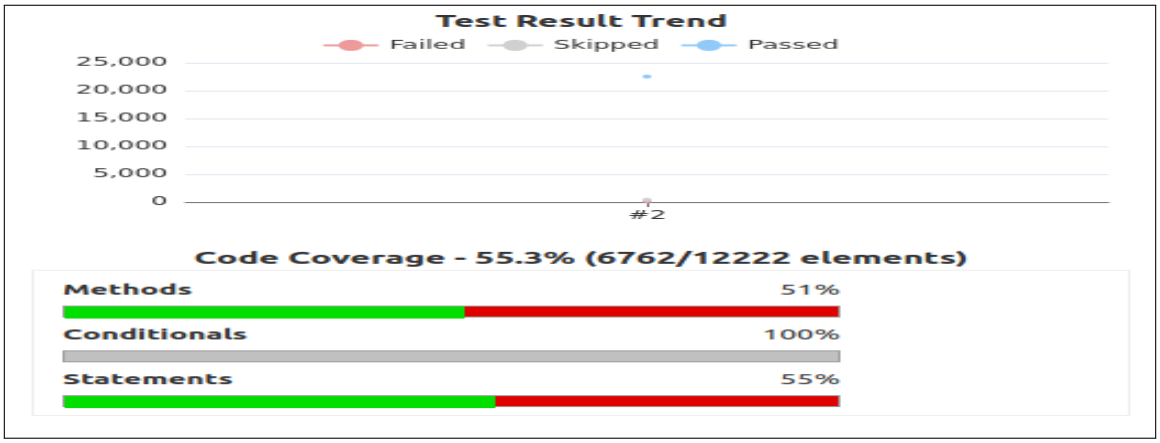


Figure 5.24: Code Coverage result in Jenkins interface

- **Quality Test:** Sonarqube is in charge of this step. It provides a comprehensive report on the source code's accuracy, including percentage of code coverage by tests, replication, bugs, and so on. The QA engineer can access this report via the Sonarqube portal.

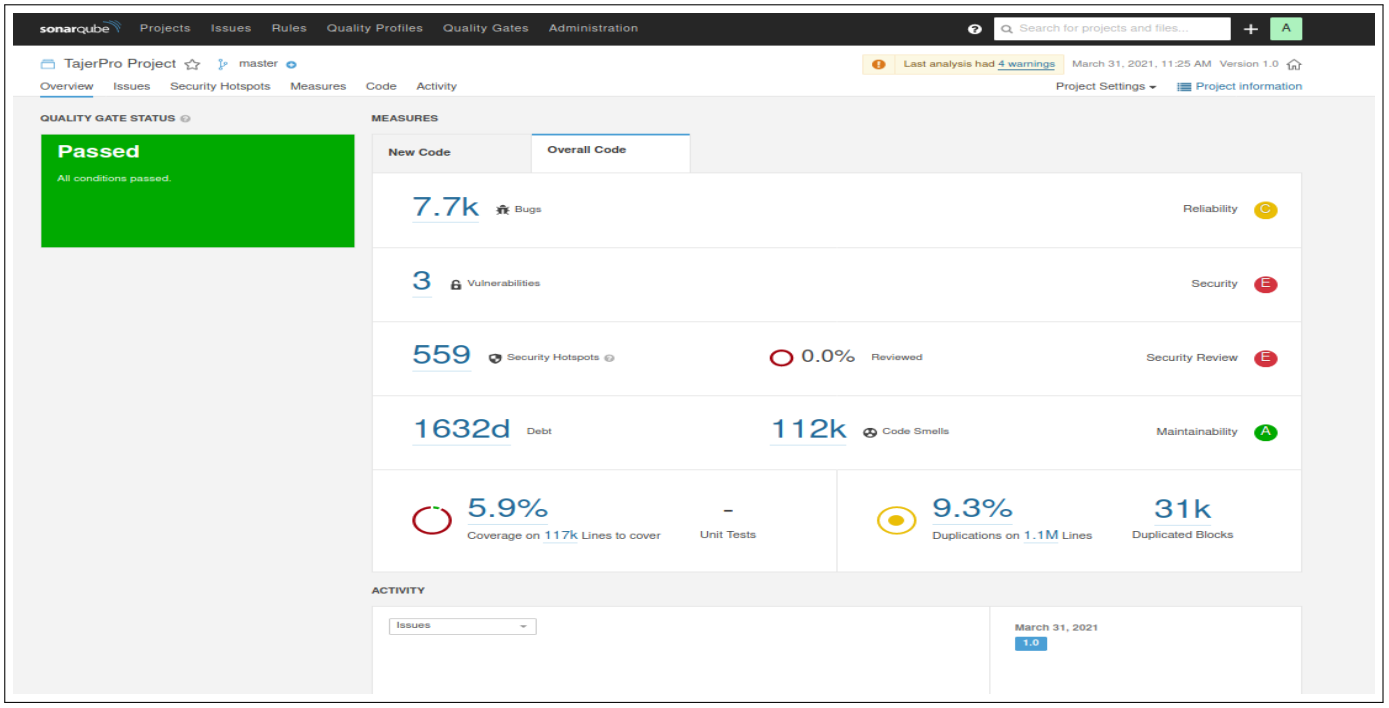


Figure 5.25: Report of Magento code in Sonarqube

- **Docker Build** The application's Docker image will be generated at this stage. Figure 5.26 depicts the command that allows this image to be created.

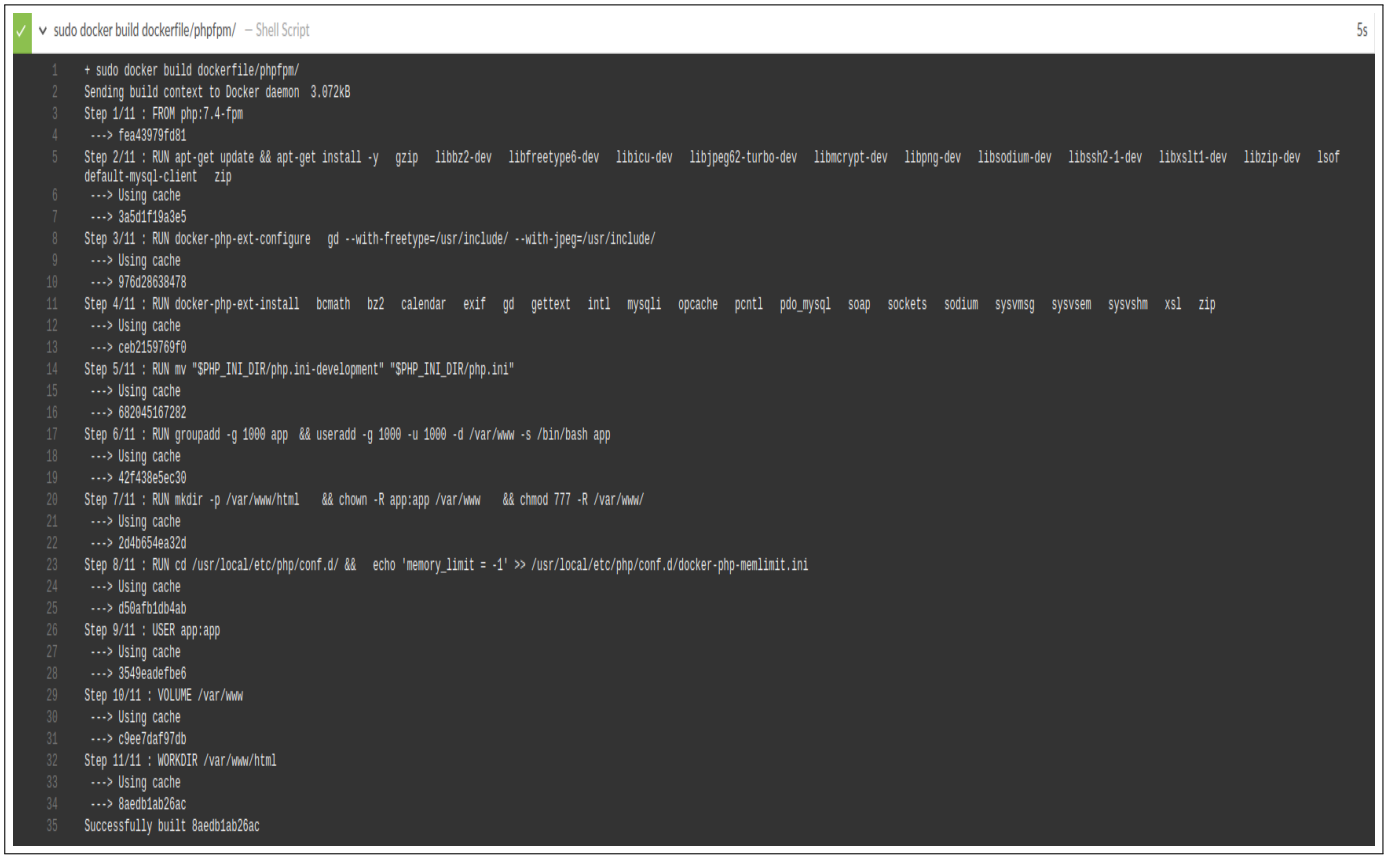


Figure 5.26: Output of building dockerfile

- **Dockerization:** In this step, we will compose our container to work with each other by running the docker-compose. Parallelly, we will make sure that our Magento project is up-to-date and running the needed command for it to properly work.



Figure 5.27: Output of running the docker containers with docker-compose

- **Functional Test:** To run a functional test we have to run a web driver with selenium to run the test generated by our Magento application , in figure 5.28 we will display a test result of one of the tests.

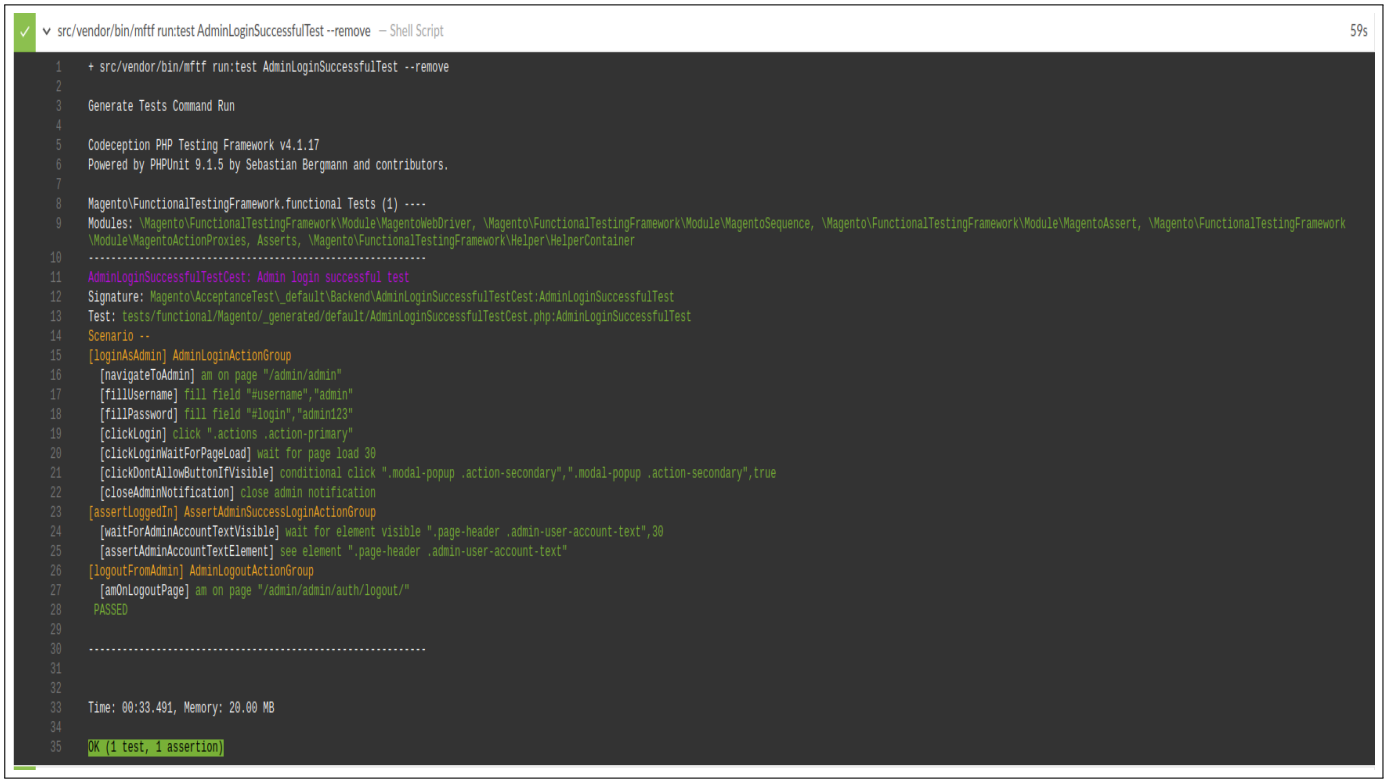


Figure 5.28: Result of a generated functional test

- **Deploy:** By deploying our project in this stage we mean that our pods are up-to-date and working fine as shown in the figure bellow. and by achieving that, we will have our final product working perfectly online.

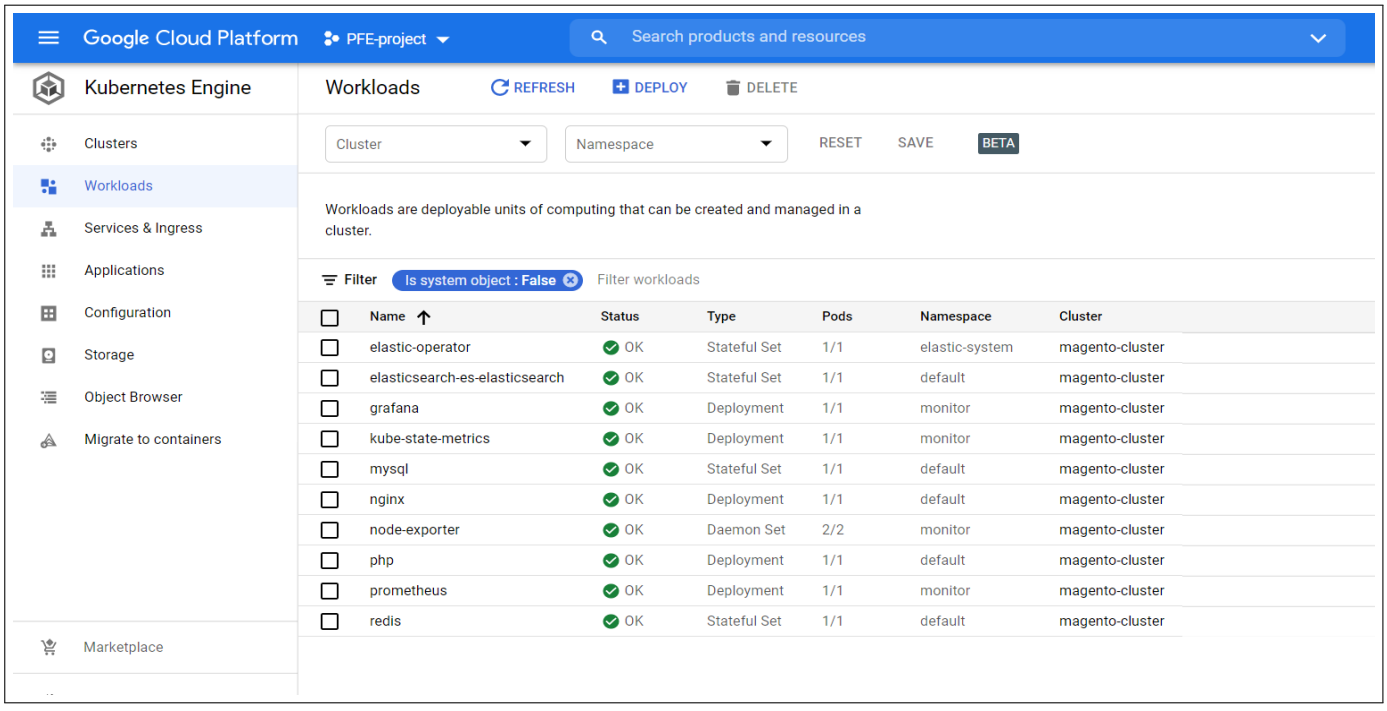


Figure 5.29: working pods in google cloud platform

- **Email notification:** when the pipeline is succecefuly built, Jenkins will send email to the DeVops engineer an email to notify him.

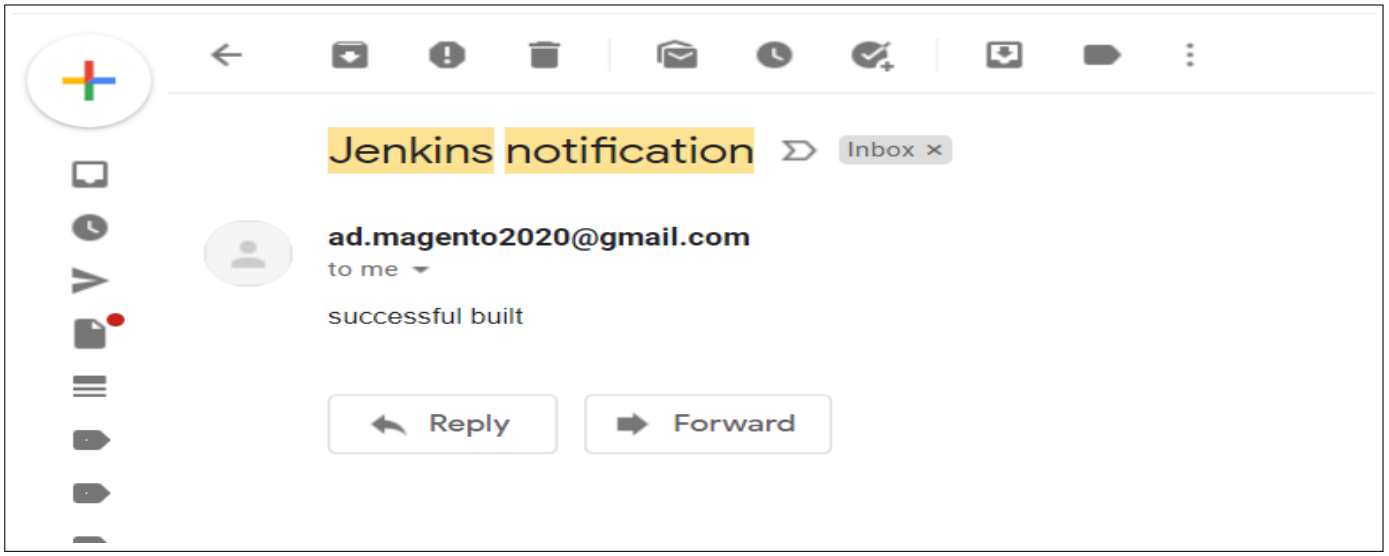


Figure 5.30: Email sent by Jenkins

Conclusion

This chapter summarizes all of the work that was completed during the internship.First and foremost, we've discussed the hardware and software environment, as well as the various tools that were used to complete this project. Then we went over the various stages of putting our final solution into action. After that, with the aid of screenshots, we have shown the incorporated functionalities. We have reached the end of our journey.Finally, snapshots were used to demonstrate the applied functionalities. The report is then concluded with a general conclusion.

General Conclusion

Effective corporate change necessitates a philosophy that transcends the conventional boundaries that isolate the organization's numerous teams. In the case of Devops, we want to pull together the development and operations departments. The basic philosophy is to facilitate cooperation and coordination among all parties engaged in the development of software and systems, with a special emphasis on process and product enhancement. In reality, Devops includes the use of automation tools, virtualization, and, most notably, continuous integration to streamline the distribution process.Continuous integration and delivery is now considered a must for everyone who wants to modernize their production activities.

The project "Automation of the Devops workflow to implement the Magento solution," which is part of the Softifi company's end-of-studies project, is a step ahead in their Devops strategy. Via a range of tools that make up the software factory, the ultimate aim is to ensure the consistency and fluidity of the development cycle. To boost overall project efficiency, this factory allows for the automation of builds, deployments, and test suites, as well as the generation of test results that are open to all team members.

We followed a set of well-thought-out steps to accomplish the project's goals. It was true that a preliminary inquiry was required first. This analysis was very effective in understanding Softifi's current operating system, outlining the company's requirements in a straightforward fashion, and recognizing the functionalities that needed to be incorporated. Second, we looked at various concepts that are essential for a thorough understanding of this project, such as Devops, continuous integration/continuous delivery , virtualization, and software testing. Then we conducted a comparative analysis in which we described the methodology we took and the specifics of the studies we conducted with each part of our method. Then, using the UML and BPMN models, we attacked the architecture aspect of the proposed solution, which helped us to model our method well enough that it is easier to interpret and execute, enabling us to verify the implementation of our solution.

After completing this project, we were able to apply our theoretical and functional knowledge of containerization, scripting, and management of Linux systems, as well as challenge them with the company's requirements.Overall, this end-of-studies initiative was a very instructive experience, as well as a chance to argue and explore diverse points of view in follow-up discussions. It's worth noting that when setting up the CI/CD setup, we ran into a number of issues and roadblocks. We ran into issues with coordination between the different components

of this system in particular.

In conclusion, while this work has achieved the majority of its stated goals, we hope to replicate it across all Magento projects and extend the automated build process to the application's deployment at the end user.

Appendix A

Dockerfile

```
FROM php:7.4-fpm

RUN apt-get update && apt-get install -y \
  gzip \
  libbz2-dev \
  libfreetype6-dev \
  libicu-dev \
  libjpeg62-turbo-dev \
  libmcrypt-dev \
  libpng-dev \
  libsodium-dev \
  libssh2-1-dev \
  libxslt1-dev \
  libzip-dev \
  lsof \
  default-mysql-client \
  zip

RUN docker-php-ext-configure \
  gd --with-freetype=/usr/include/ --with-jpeg=/usr/include/

RUN docker-php-ext-install \
  bcmath \
  bz2 \
  calendar \
  exif \
  gd \
  gettext \
  intl \
  mysqli \
  opcache \
  pcntl \
  pdo_mysql \
  soap \
  sockets \
  sodium \
  sysvmsg \
  sysvsem \
  sysvshm \
  xsl \
  zip
RUN mv "$PHP_INI_DIR/php.ini-development" "$PHP_INI_DIR/php.ini"
RUN groupadd -g 1000 app \
 && useradd -g 1000 -u 1000 -d /var/www -s /bin/bash app
RUN mkdir -p /var/www/html \
   && chown -R app:app /var/www \
   && chmod 777 -R /var/www/
RUN cd /usr/local/etc/php/conf.d/ && \
  echo 'memory_limit = -1' >> /usr/local/etc/php/conf.d/docker-php-memlimit.ini

USER app:app
VOLUME /var/www
WORKDIR /var/www/html
```

Docker-compose.yml

```yaml
version: '3.7'
services:

  app:
    image: nginx:latest
    container_name: nginx-container
    networks:
      - magento-network
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
      - ./src:/var/www/html


  phpfpm:
    build: ./dockerfile/phpfpm
    container_name: php-container
    networks:
      - magento-network
    volumes:
      - ./src:/var/www/html


  mysql:
        image: mysql:5.7
        container_name: mysql-container
        networks:
            - magento-network
        ports:
            - "3306:3306"
        volumes:
            - ./mysql:/var/lib/mysql
        environment:
            MYSQL_ROOT_PASSWORD: password
            MYSQL_USER: docker_user
            MYSQL_PASSWORD: root
            MYSQL_DATABASE: magento_demo

  elasticsearch:
    image: elasticsearch:7.10.1
    container_name: elasticsearch-container
    networks:
      - magento-network
    ports:
      - "9300:9300"
      - "9200:9200"
    environment:
      discovery.type: single-node

networks:
    magento-network:
volumes:
  mysql:
```

Appendix B

Php-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php
  labels:
    tier: backend

spec:
  replicas: 1
  selector:
    matchLabels:
      app: php
      tier: backend

  template:
    metadata:
      labels:
        app: php
        tier: backend
    spec:

      volumes:
      - name: dir
        persistentVolumeClaim:
          claimName: dir
      containers:
      - name: php
        image: nessrinehafi/tajerpro_phpfpm:latest
        imagePullPolicy: IfNotPresent
        securityContext:
          allowPrivilegeEscalation: false
          runAsUser: 0
        volumeMounts:
        - name: dir
          mountPath: /var/www/
```

Php-service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: php
  labels:
    tier: backend

spec:
  selector:
    app: php
    tier: backend

  ports:
  - protocol: TCP
    port: 9000
```

dir-pvc.yaml

```yaml
    apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dir
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
```

nginx-deployment.yaml

```yaml
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
      tier: backend

  template:
    metadata:
      labels:
        app: nginx
        tier: backend

    spec:
      volumes:
      - name: dir
        persistentVolumeClaim:
          claimName: dir
      - name: config
        configMap:
          name: nginx-config
          items:
          - key: config
            path: site.conf
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
        volumeMounts:
        - name: dir
          mountPath: /var/www/
        - name: config
          mountPath: /etc/nginx/conf.d
```

Bibliography

[1] "Softifi," https://softifi.com/en/home/, consulted the 4 Mars 2021.

[2] "Bridge between dev and ops," https://www.insightssuccess.com/devops-bridging-the-gap-between-dev consulted the 5 Mars 2021.

[3] "Devops," http://ijcsit.com/docs/Volume206/vol6issue03/ijcsit2015060382.pdf, consulted the 5 Mars 2021.

[4] "Ci/cd," https://www.infoworld.com/article/3271126/what-is-cicd-continuous-integration-and-continu consulted the 10 Mars 2021.

[5] "Benefits of ci/cd," https://www.lambdatest.com/blog/benefits-of-ci-cd/, consulted the 10 Mars 2021.

[6] "containerization," https://www.citrix.com/solutions/app-delivery-and-security/what-is-containerizat consulted the 10 Mars 2021.

[7] "Containers," https://www.docker.com/resources/what-container, consulted the 10 Mars 2021.

[8] "Rest," https://searchitoperations.techtarget.com/definition/Docker, consulted the 10 Mars 2021.

[9] "Docker image," https://jfrog.com/knowledge-base/a-beginners-guide-to-understanding-and-building- consulted the 10 Mars 2021.

[10] "Dockerfile," https://www.edureka.co/community/48689/what-docker-file-how-do-create-docker-imag consulted the 10 Mars 2021.

[11] "Docker compose," https://www.educative.io/blog/docker-compose-tutorial, consulted the 10 Mars 2021.

[12] "kubernetes core concepts," https://medium.com/onfido-tech/container-orchestration-with-kubernete consulted the 11 Mars 2021.

[13] "White box testing," https://www.guru99.com/white-box-testing.html, consulted the 11 Mars 2021.

[14] “Black box testing,” https://www.guru99.com/black-box-testing.html, consulted the 11 Mars 2021.

[15] “Bitbucket,” https://bitbucket.org/product/en/version-control-software, consulted the 12 Mars 2021.

[16] “monitoring,” https://www.blameless.com/blog/how-to-choose-monitoring-tools, consulted the 13 Mars 2021.

[17] “grafana,” https://www.8bitmen.com/what-is-grafana-why-use-it-everything-you-should-know-about- consulted the 14 Mars 2021.

[18] “monitoring tools,” https://mindmajix.com/list-of-bi-or-monitoring-devops-tools, consulted the 14 Mars 2021.

[19] “Uml,” https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/, consulted the 20 Mars 2021.

[20] “Bpmn,” https://www.visual-paradigm.com/tutorials/how-to-create-bpmn-diagram/, consulted the 20 Mars 2021.

[21] “workflow,” https://deploybot.com/blog/using-multiple-environments-to-improve-your-development- consulted the 23 Mars 2021.

ملخّص

يندرج هذا العمل ضمن مشروع نهاية دراستنا المقام بشركة Softifi، الهدمن هذا المشروع هو تطوير وتقديم تكامل مستمر وخط أنابيب تسليم مستمر يدعم تجميع كود المصدر وتحليله وإطلاق الاختبار وإنشاء صورة Docker واستخدام Containers والنشر مع Kubernetes.

كلمات مفتيح : Softifi، Jenkins، Docker، Docker Compose، Kubernetes، Magento، DeVops، Selenium

Résumé

Notre travail intitulé "Automatisation du workflow devops pour intégrer la solution Magento" dans le cadre de notre projet de fin d'études à Softifi. L'objectif de ce projet est de développer et d'introduire un pipeline d'intégration continue, de livraison continue qui prend en charge la compilation et l'analyse du code source, le lancement des tests, la génération d'images Docker, l'utilisation de conteneurs et le déploiement avec Kubernetes.

Mots clés : Softifi, Jenkins, Docker, Docker Compose, Kubernetes, Magento, DeVops, Selenium

Abstract

Our work entitled "Automation of the devops workflow to integrate the Magento solution" as part of our final year project at Softifi. The aim of this project is to develop and introduce a continuous integration, continuous delivery pipeline that supports compiling and analyzing source code, launching tests, generating Docker images, managing containers, and deploying with Kubernetes.

Keywords : Softifi, Jenkins, Docker, Docker Compose, Kubernetes, Magento, DeVops, Selenium